

LECTURE 15

Cache-Oblivious Algorithms

Xuhao Chen

March 26, 2025



Acknowledgment

A presentation by Matteo Frigo inspired some of these slides.

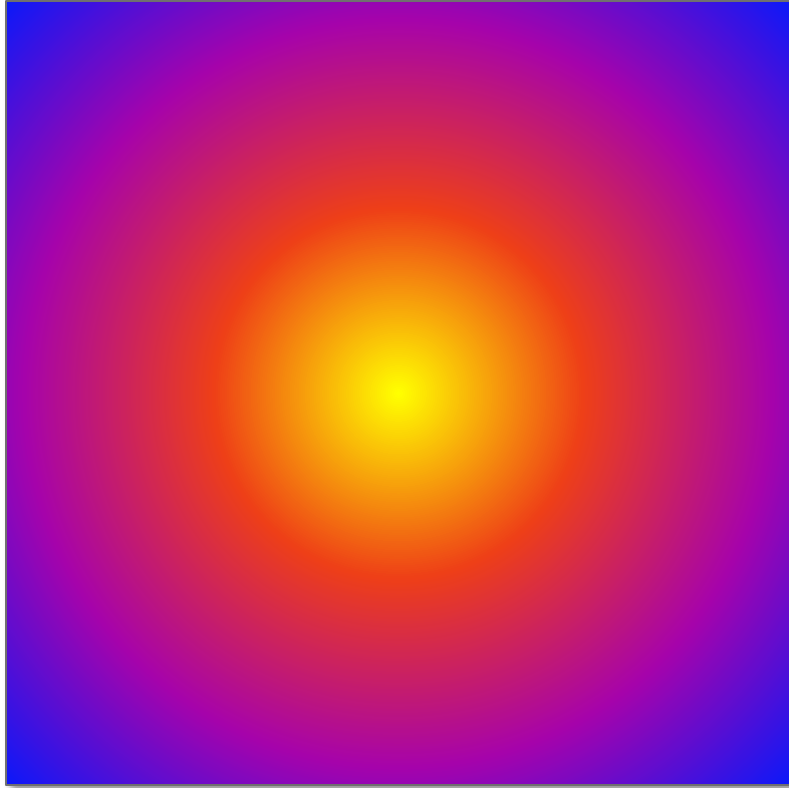
Outline

- Simulation of Heat Diffusion → Stencil Computation
- Cache-oblivious Stencil Computation
- Parallelizing Cache-oblivious Stencil Computation
- Summary

SIMULATION OF HEAT DIFFUSION



Heat Diffusion



2D heat equation*

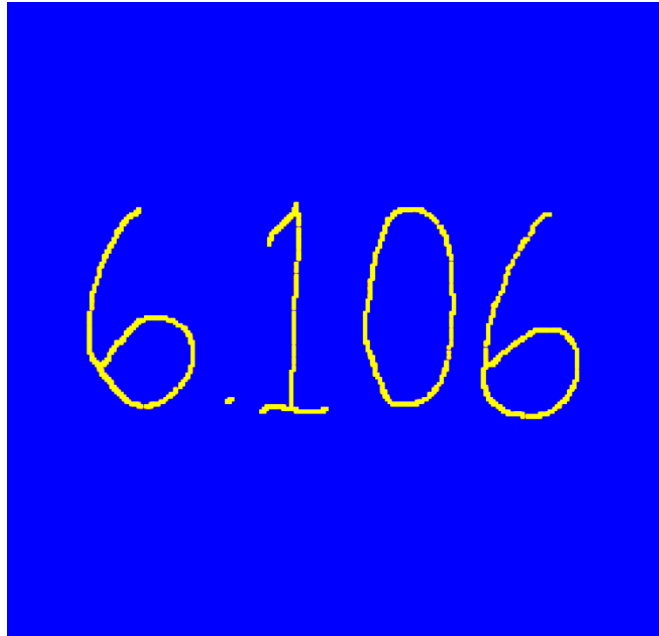
The **heat function** $u(t, x, y)$ is the heat at time t of a point (x, y) .

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

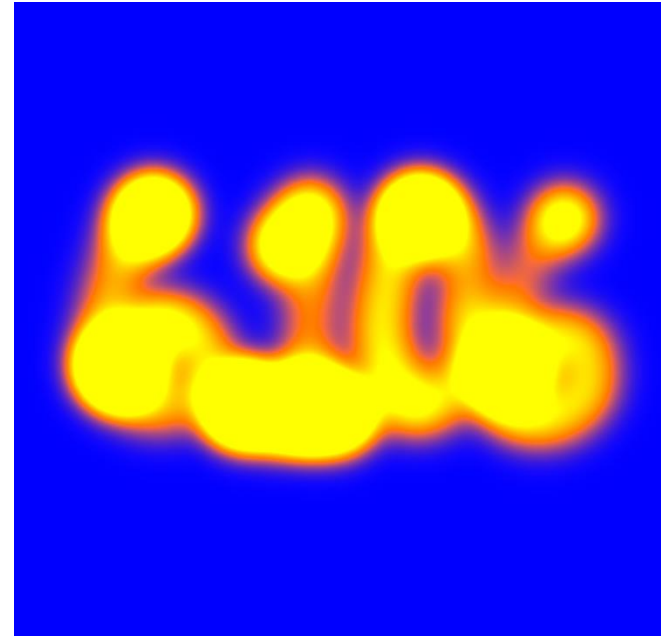
α is the **thermal diffusivity**.

* originally formulated by Joseph Fourier in 1807

2D Heat-Diffusion Simulation



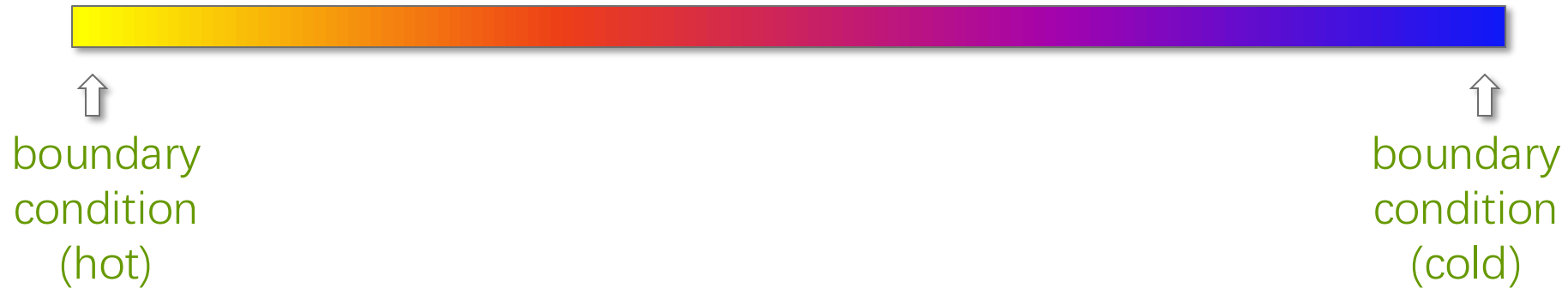
Before



After

1D Heat Equation

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$



Finite-Difference Method

The famous Swiss mathematician **Leonhard Euler** (1707–1783) invented the **finite-difference method** around 1768.

We owe Euler credit for the notations

- $f(x)$ for a function,
- e for the base of the natural logarithm,
- i for the square root of -1 ,
- π for the area of a unit circle,
- $\sum$ for summation,
- Δ for finite differences.



Finite-Difference Approximation

$$\frac{\partial}{\partial t} u(t, x) \approx \frac{u(t + \Delta t, x) - u(t, x)}{\Delta t},$$

$$\frac{\partial}{\partial x} u(t, x) \approx \frac{u(t, x) - u(t, x - \Delta x)}{\Delta x},$$

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

1D heat equation

$$\frac{\partial^2}{\partial x^2} u(t, x) \approx \frac{\frac{\partial}{\partial x} u(t, x + \Delta x) - \frac{\partial}{\partial x} u(t, x)}{\Delta x}$$

$$\approx \frac{\frac{u(t, x + \Delta x) - u(t, x)}{\Delta x} - \frac{u(t, x) - u(t, x - \Delta x)}{\Delta x}}{\Delta x}$$

$$\approx \frac{u(t, x + \Delta x) - 2u(t, x) + u(t, x - \Delta x)}{(\Delta x)^2}.$$

Discretized Heat Equation

$$\frac{u(t + \Delta t, x) - u(t, x)}{\Delta t} = \alpha \left(\frac{u(t, x + \Delta x) - 2u(t, x) + u(t, x - \Delta x)}{(\Delta x)^2} \right)$$

Now, put the one term involving $t + \Delta t$ on the left and the other terms involving just t on the right:

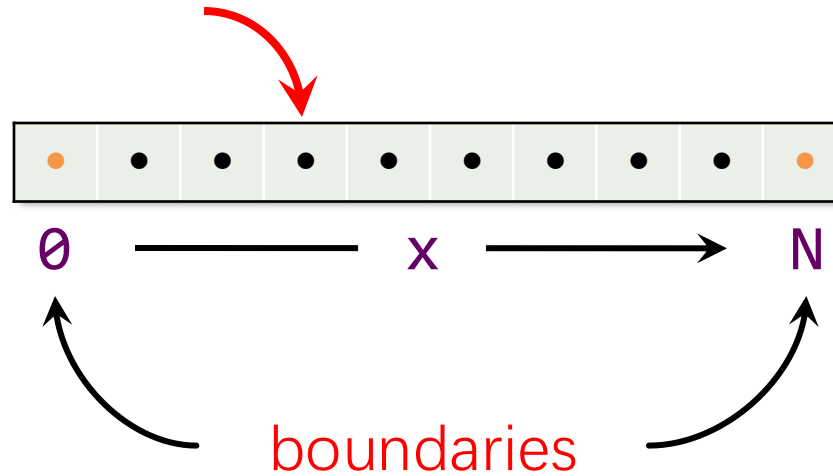
$$u(t + \Delta t, x) = u(t, x) + \alpha \Delta t \left(\frac{u(t, x + \Delta x) - 2u(t, x) + u(t, x - \Delta x)}{(\Delta x)^2} \right)$$

Assuming that $\Delta t = 1$ and $\Delta x = 1$, we obtain the following code for the **update rule**:

```
u[t+1][x] = u[t][x] + ALPHA * (u[t][x+1] - 2*u[t][x] + u[t][x-1]);
```

3-Point Stencil

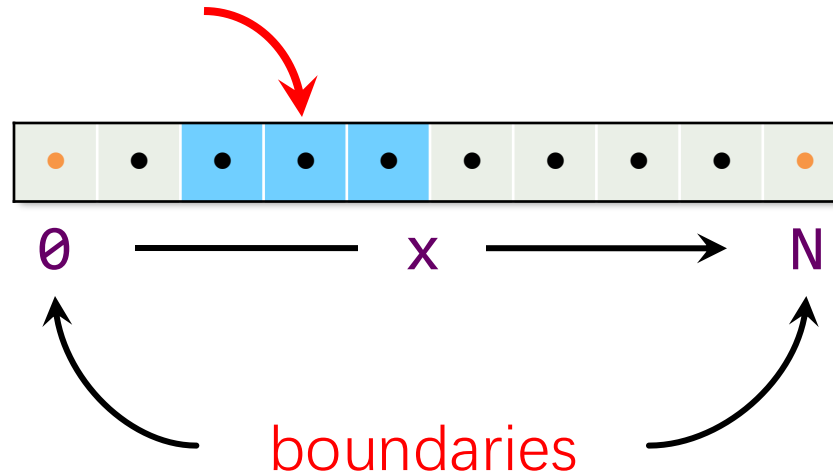
```
u[t+1][x] = u[t][x] + ALPHA * (u[t][x+1] - 2*u[t][x] + u[t][x-1]);
```



A **stencil computation** updates each point in an array by a fixed pattern, called a **stencil**.

3-Point Stencil

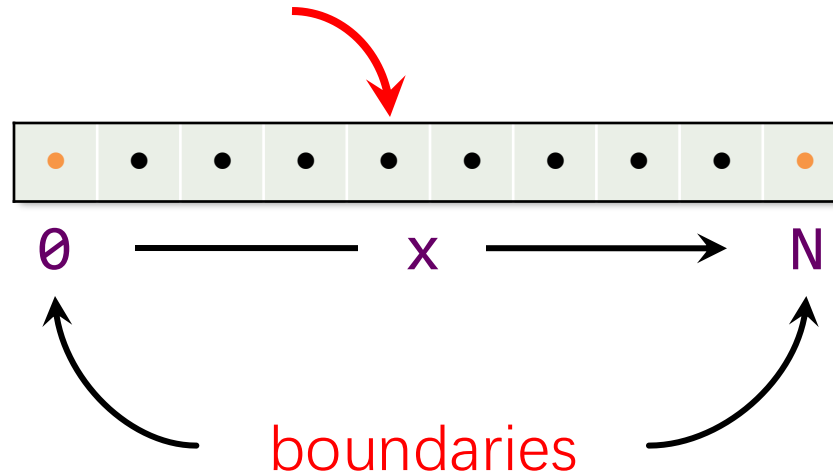
```
u[t+1][x] = u[t][x] + ALPHA * (u[t][x+1] - 2*u[t][x] + u[t][x-1]);
```



A **stencil computation** updates each point in an array by a fixed pattern, called a **stencil**.

3-Point Stencil

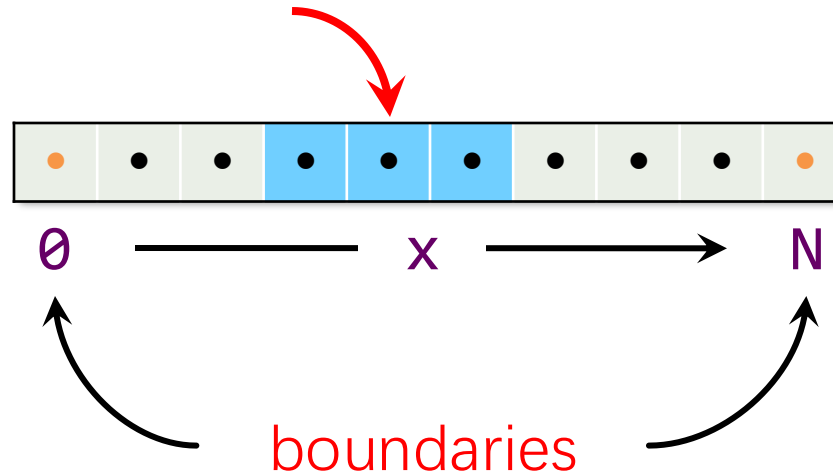
```
u[t+1][x] = u[t][x] + ALPHA * (u[t][x+1] - 2*u[t][x] + u[t][x-1]);
```



A **stencil computation** updates each point in an array by a fixed pattern, called a **stencil**.

3-Point Stencil

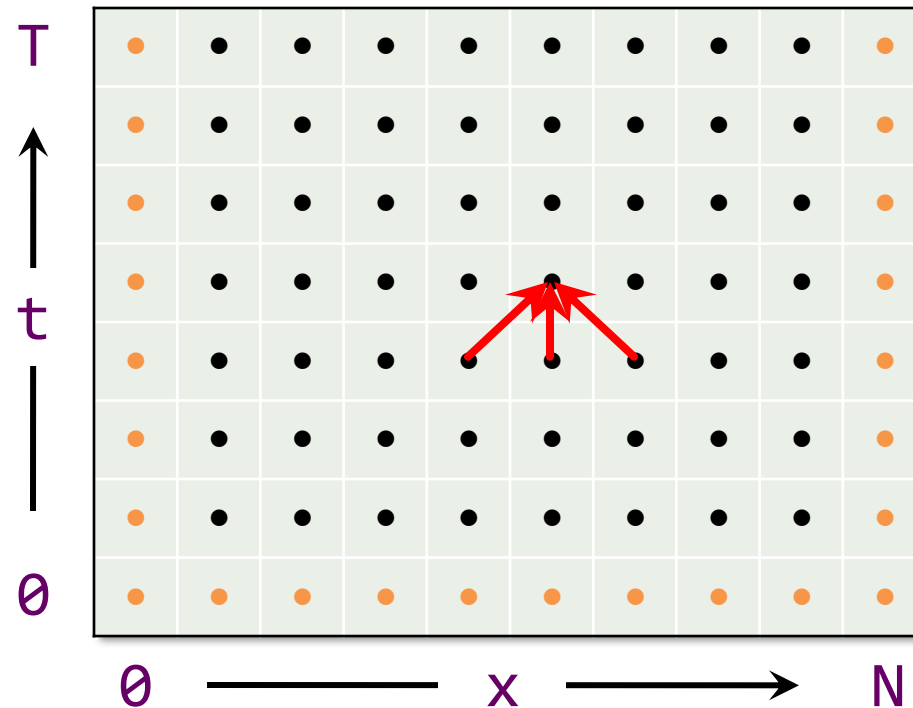
```
u[t+1][x] = u[t][x] + ALPHA * (u[t][x+1] - 2*u[t][x] + u[t][x-1]);
```



A **stencil computation** updates each point in an array by a fixed pattern, called a **stencil**.

3-Point Stencil

$$u[t+1][x] = u[t][x] + \text{ALPHA} * (u[t][x+1] - 2*u[t][x] + u[t][x-1]);$$

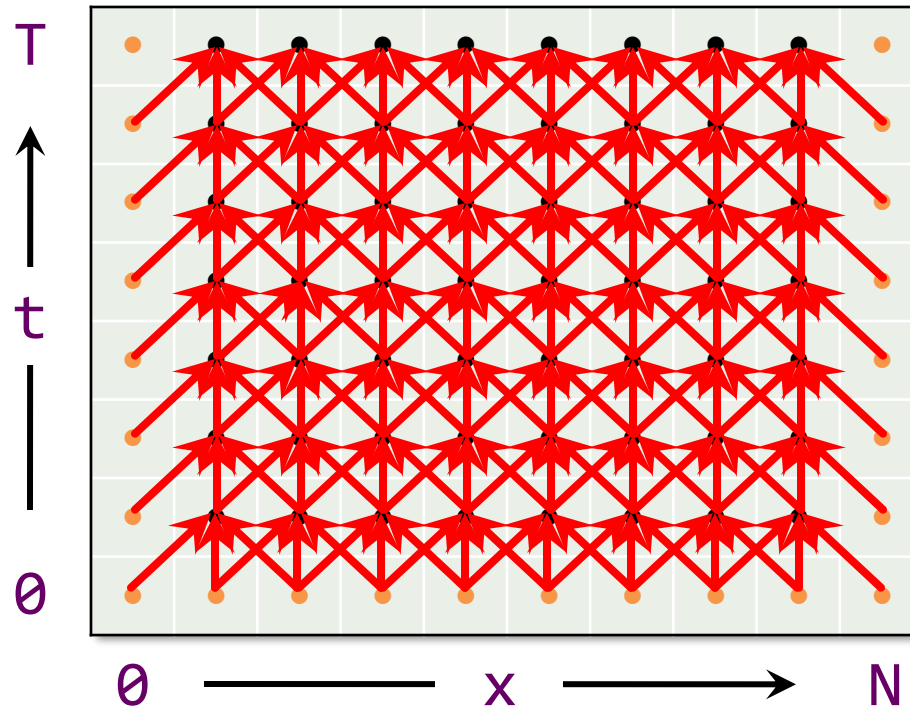


A **stencil computation** updates each point in an array by a fixed pattern, called a **stencil**.

iteration space

3-Point Stencil

$$u[t+1][x] = u[t][x] + \text{ALPHA} * (u[t][x+1] - 2*u[t][x] + u[t][x-1]);$$



A **stencil computation** updates each point in an array by a fixed pattern, called a **stencil**.

iteration
space

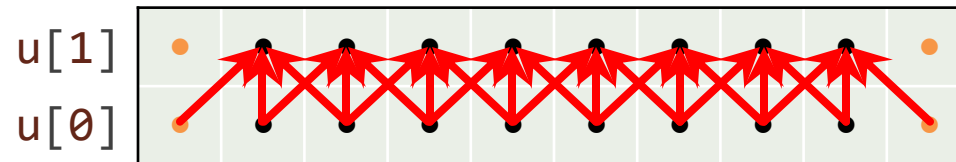
3-Point Stencil Code

```
double u[2][N]; // even-odd trick

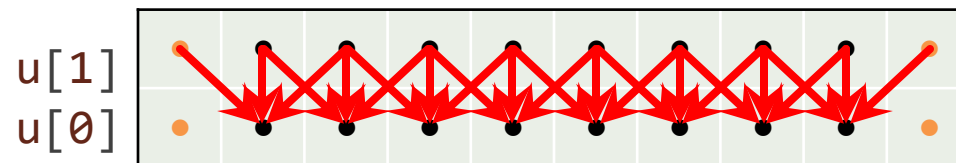
static inline double kernel(double * w) {
    return w[0] + ALPHA * (w[-1] - 2*w[0] + w[1]);
}

for (size_t t = 1; t < T-1; ++t) { // time loop
    for(size_t x = 1; x < N-1; ++x) // space loop
        u[(t+1)%2][x] = kernel( &u[t%2][x] );
}
```

Even time step



Odd time step



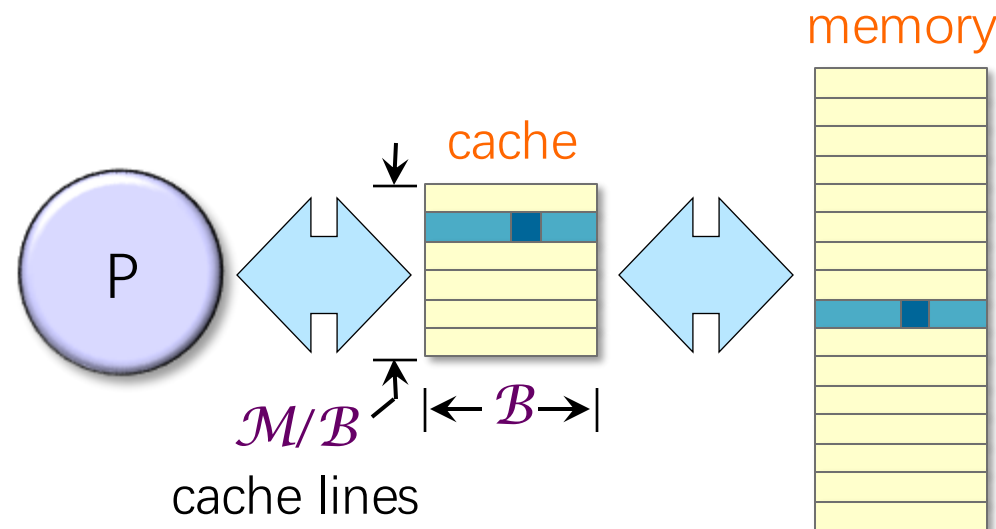
CACHE-OBLIVIOUS STENCIL COMPUTATIONS



Recall: Ideal-Cache Model

Parameters

- Two-level hierarchy
- Cache size of $\mathcal{M}$ bytes
- Cache-line size of $\mathcal{B}$ bytes
- Fully associative
- Optimal omniscient replacement, or LRU.



Performance Measures

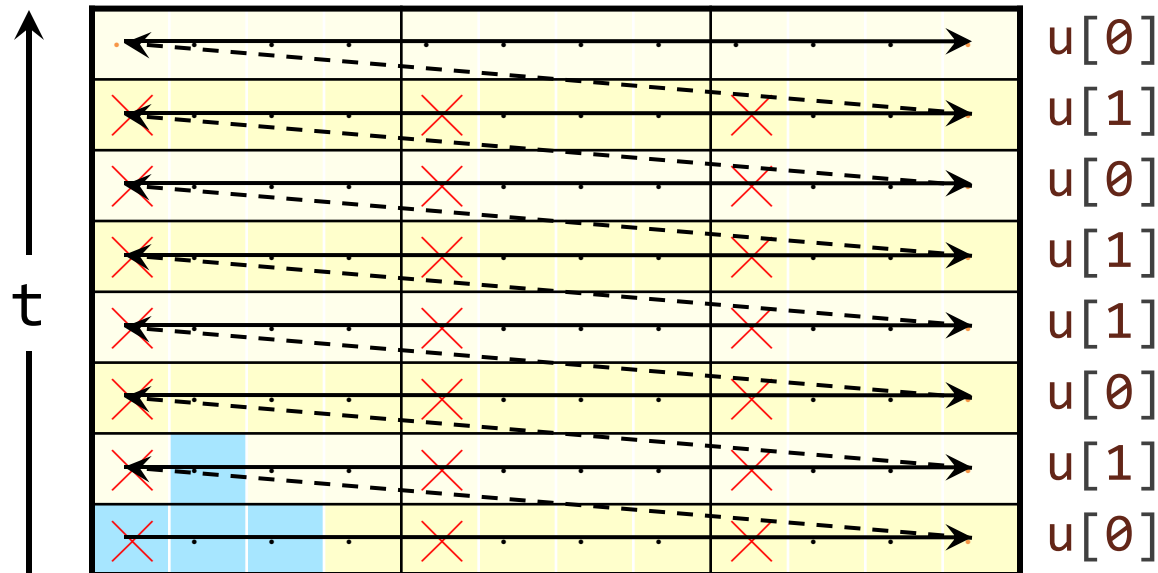
- **work** T_1 (ordinary running time)
- **cache misses** Q

Cache Behavior of Looping

```
double u[2][N]; // even-odd trick

static inline double kernel(double * w) {
    return w[0] + ALPHA * (w[-1] - 2*w[0] + w[1]);
}

for (size_t t = 0; t < T-1; ++t) { // time loop
    for (size_t x = 1; x < N-1; ++x) // space loop
        u[(t+1)%2][x] = kernel( &u[t%2][x] );
}
```



Assume that $N > M$ and
we use LRU replacement
Then $Q = \Theta(NT/B)$.

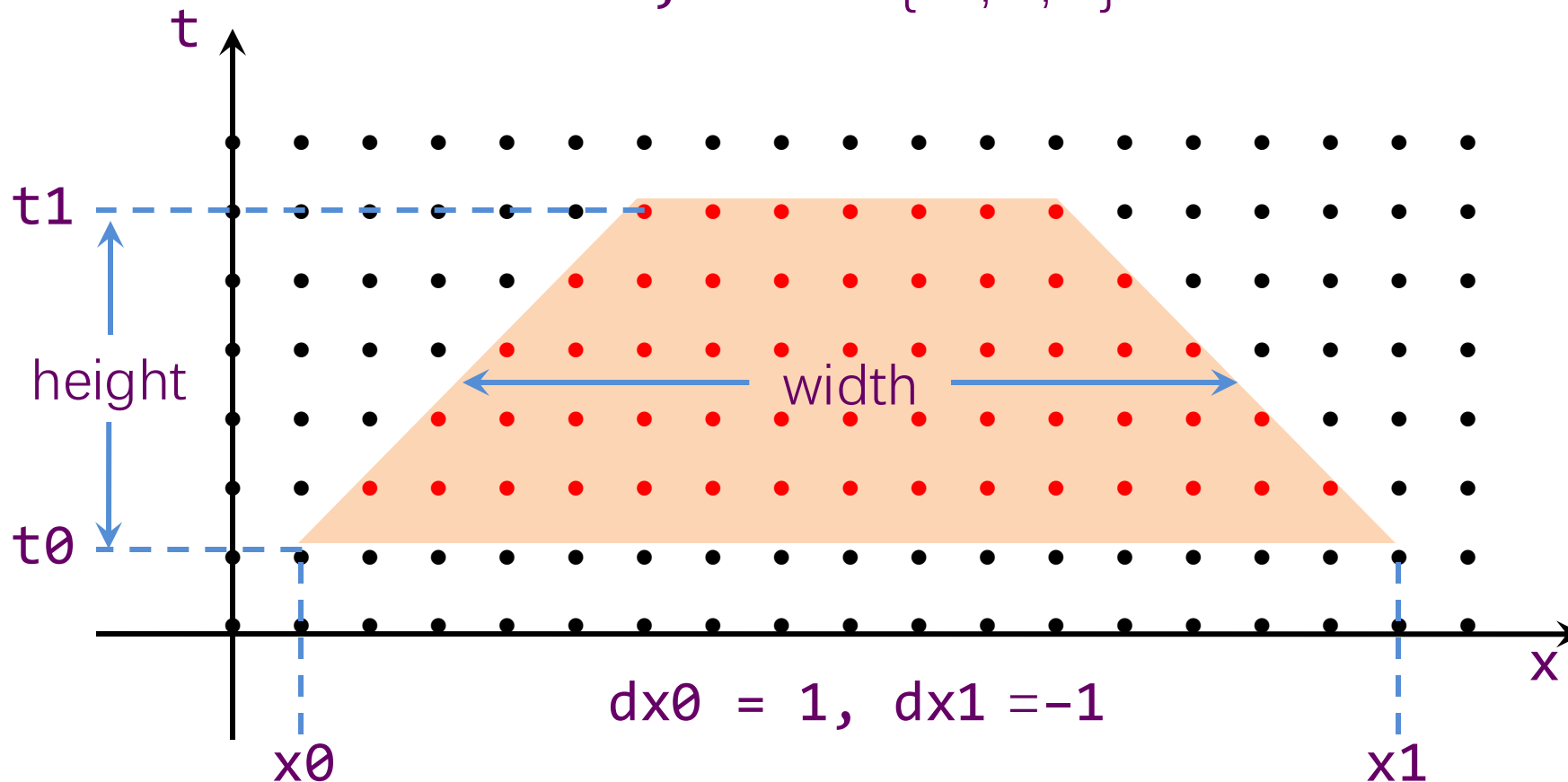
Cache-Oblivious 3-Point Stencil

Recursively traverse trapezoidal regions of space-time points (t, x) such that

$$t_0 \leq t < t_1$$

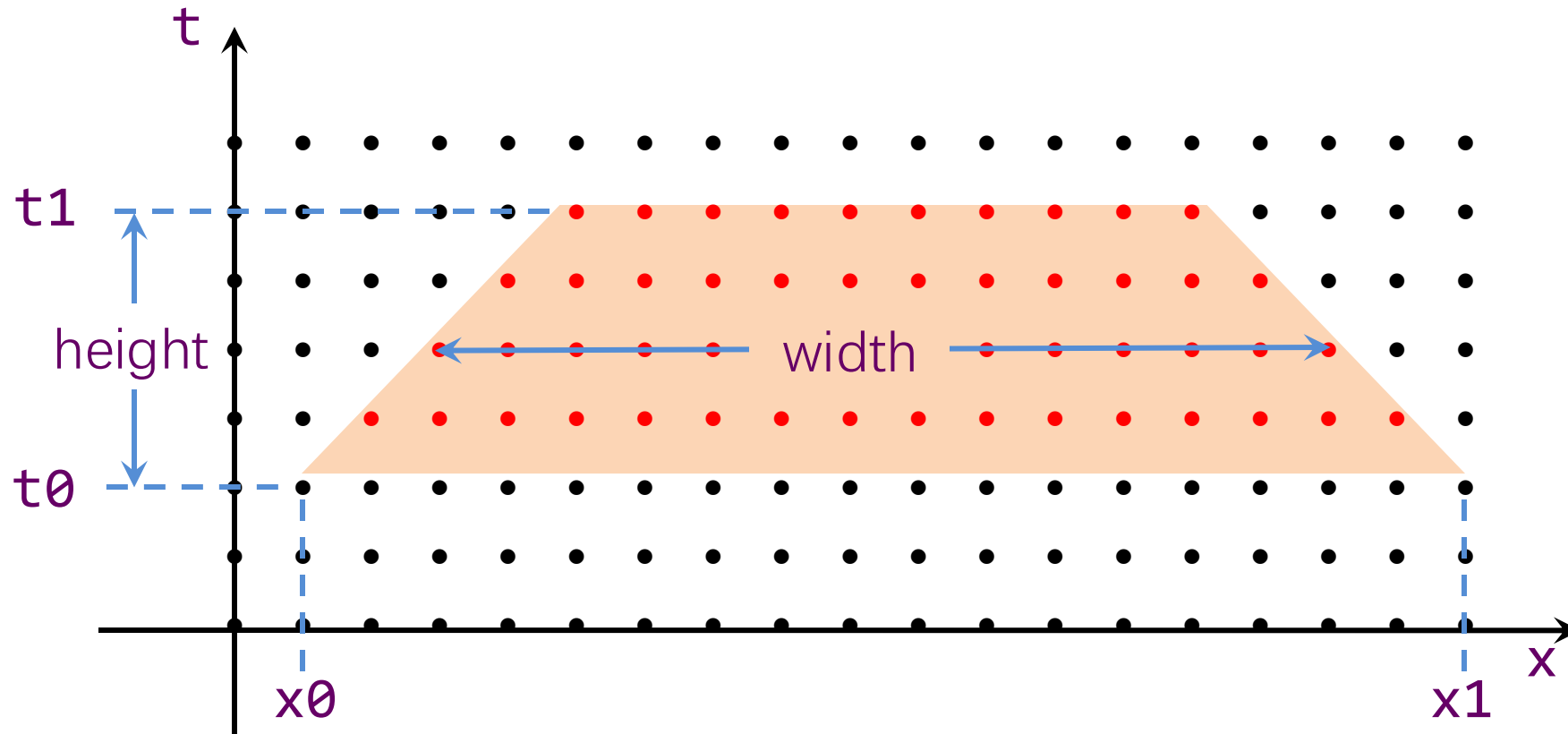
$$x_0 + dx_0(t - t_0) \leq x < x_1 + dx_1(t - t_0)$$

$$dx_0, dx_1 \in \{-1, 0, 1\}$$



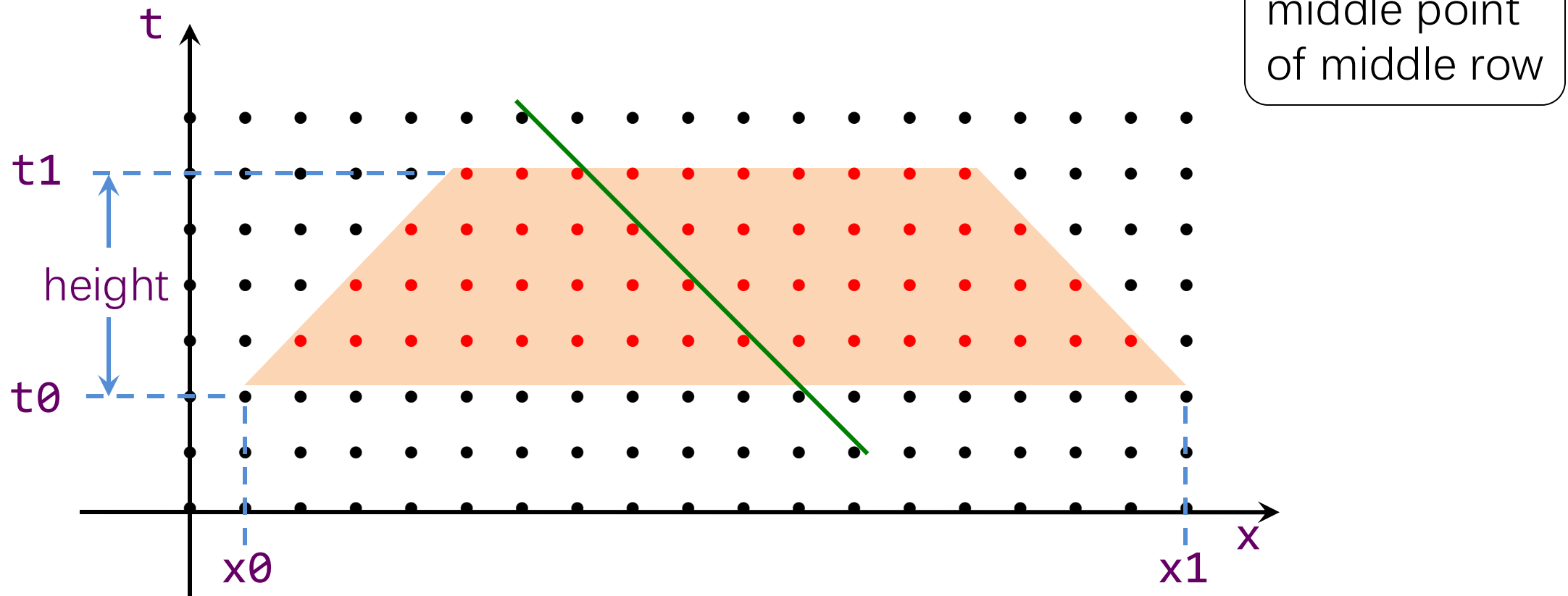
Squat Trapezoid: Space Cut

- If $\text{width} \geq 2 \cdot \text{height}$, cut the trapezoid



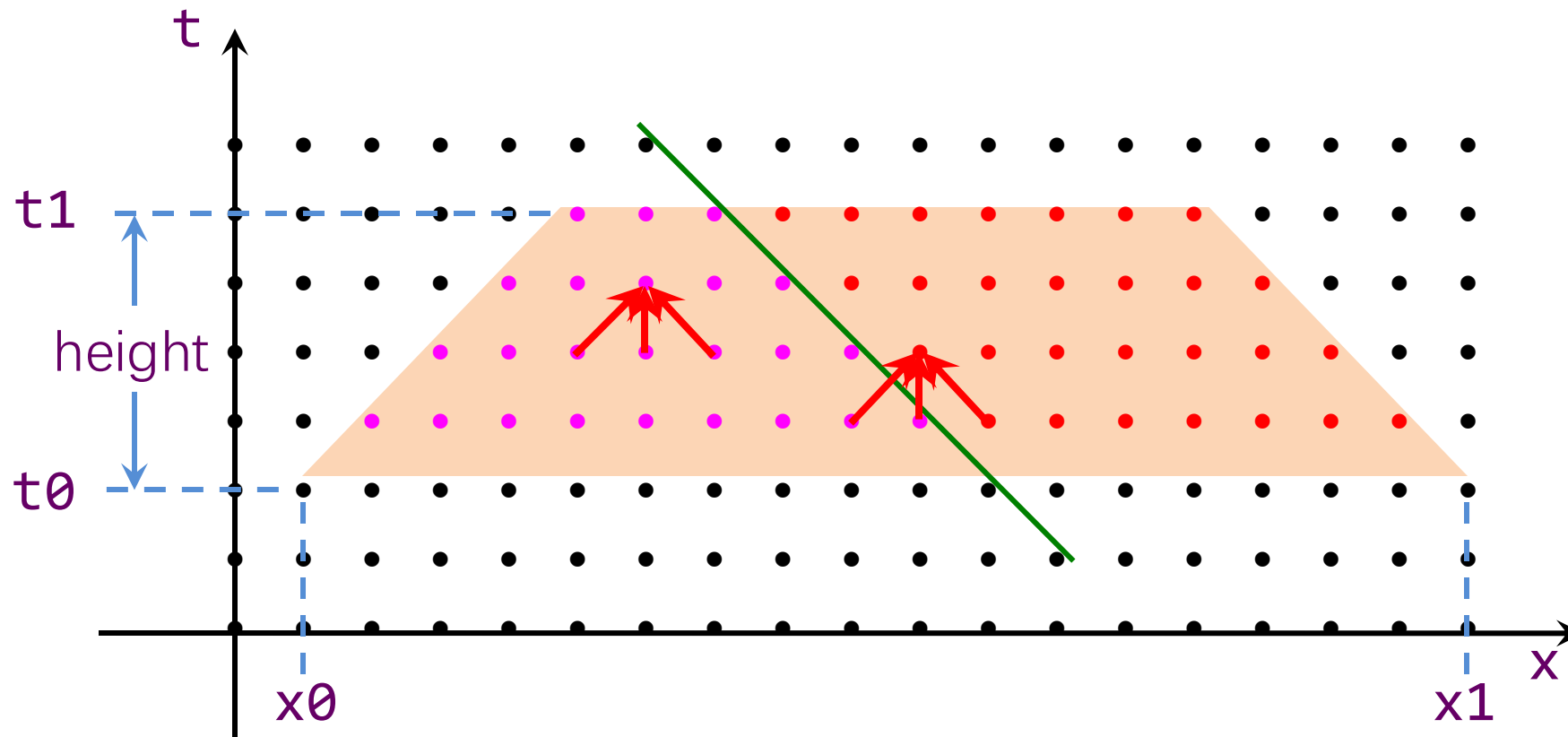
Squat Trapezoid: Space Cut

- If $\text{width} \geq 2 \cdot \text{height}$, cut the trapezoid with a line of slope -1 through the center



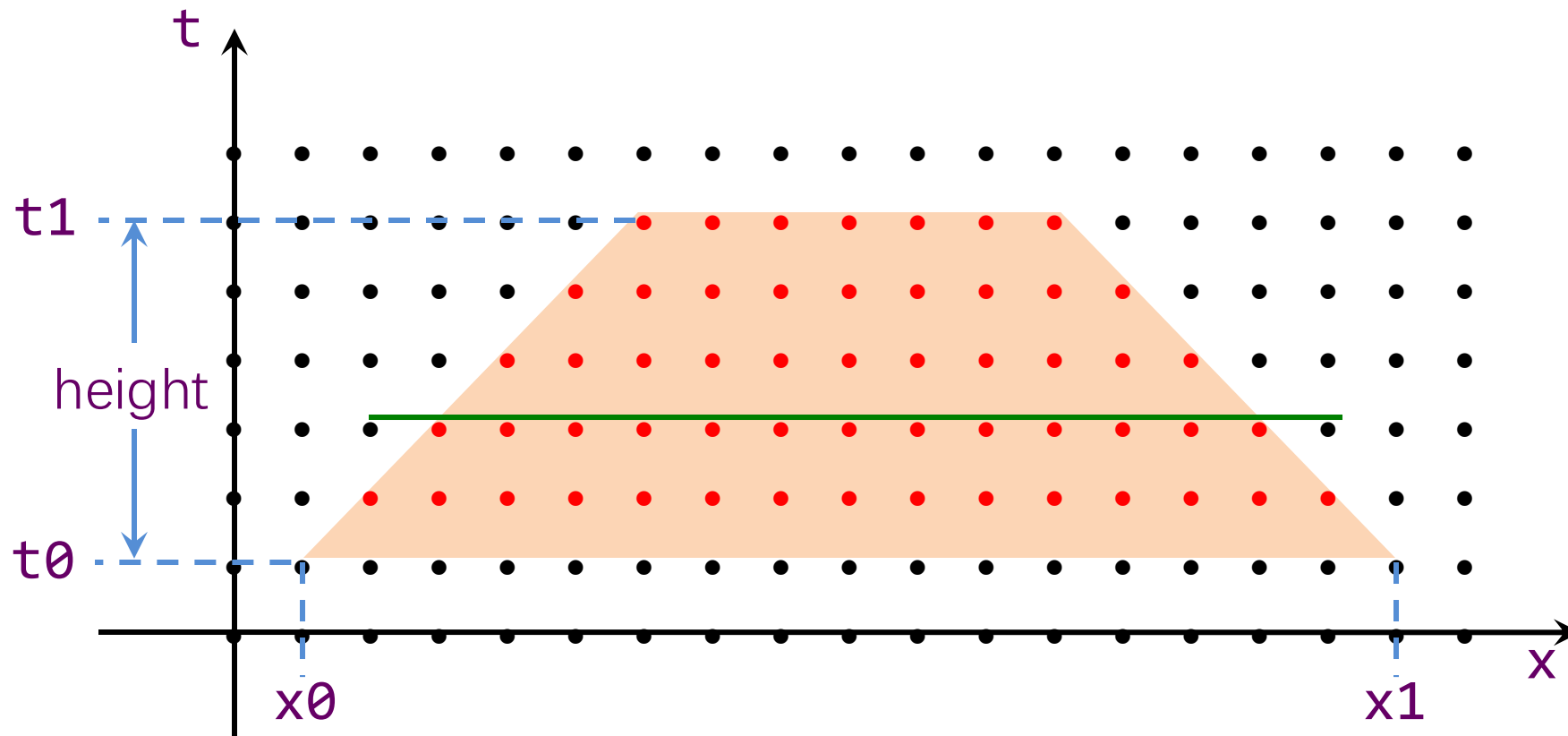
Squat Trapezoid: Space Cut

- If $\text{width} \geq 2 \cdot \text{height}$, cut the trapezoid with a line of slope -1 through the center
- Traverse the trapezoid on the **left first**, and then the one on the right.



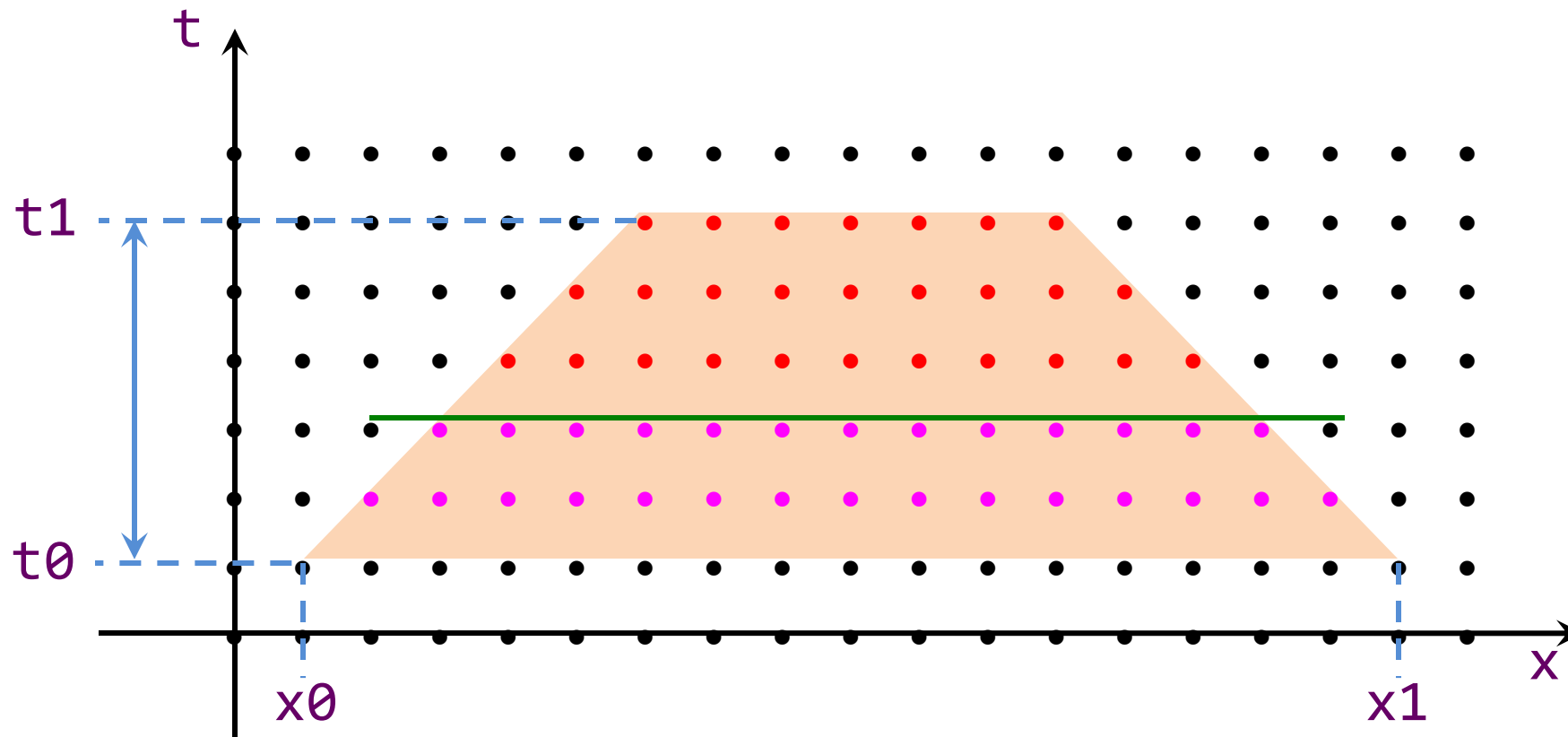
Tall Trapezoid: Time Cut

- If $\text{width} < 2 \cdot \text{height}$, cut the trapezoid with a horizontal line through the center



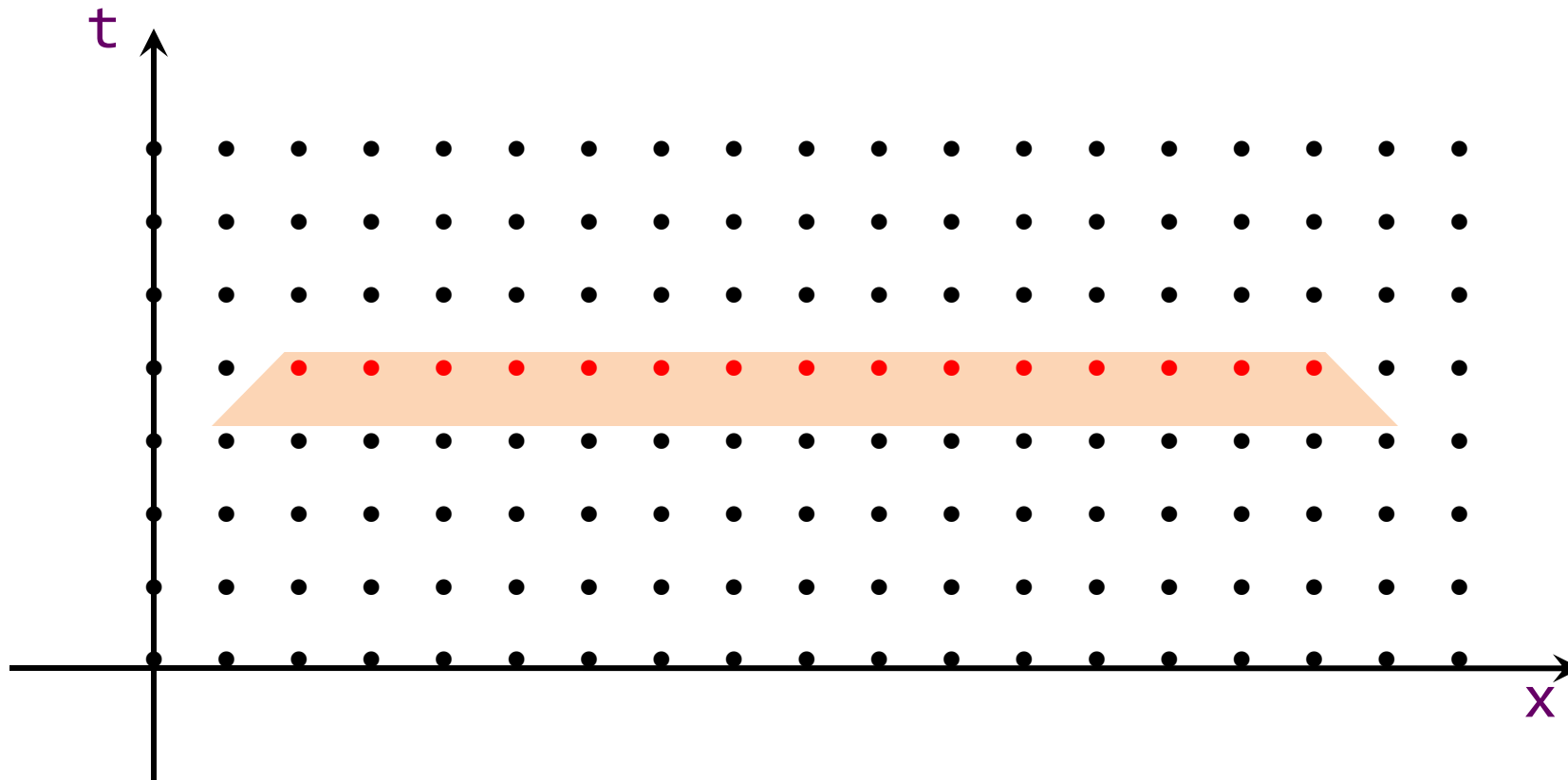
Tall Trapezoid: Time Cut

- If $\text{width} < 2 \cdot \text{height}$, cut the trapezoid with a horizontal line through the center
- Traverse the **bottom** trapezoid **first**, and then the top one



Base Case

- If $\text{height} = 1$, compute all space-time points in the trapezoid.
- Any order of computation is valid, since no point depends on another

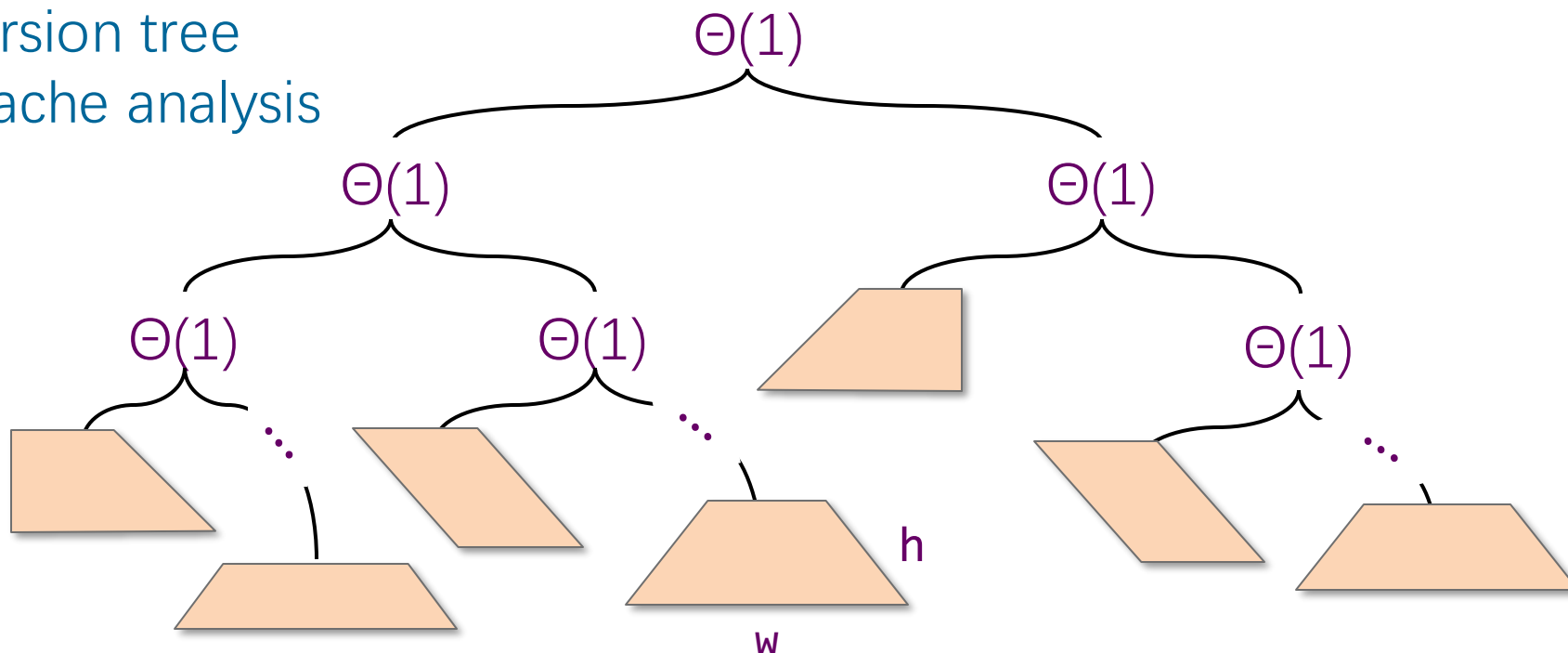


C Implementation

```
void trapezoid(int64_t t0, int64_t t1, //time start and end
              int64_t x0, int64_t dx0, //left pt of base & "slope"
              int64_t x1, int64_t dx1) { //rt pt of base & "slope"
    int64_t h = t1 - t0; //trapezoid height
    if (h == 1) { //base case
        for (int64_t x = x0; x < x1; x++)
            u[t1%2][x] = kernel( &u[t0%2][x] ); //same as in looping
    } else if (h > 1) {
        if (2*(x1 - x0) + (dx1 - dx0) * h >= 4*h) { //space cut
            int64_t xm = (2*(x0 + x1) + (dx0 + dx1 + 2)*h) / 4;
            trapezoid(t0, t1, x0, dx0, xm, -1); //left
            trapezoid(t0, t1, xm, -1, x1, dx1); //right
        } else { //time cut
            int64_t half_h = h / 2;
            trapezoid(t0, t0 + half_h, x0, dx0, x1, dx1); //bottom
            trapezoid(t0 + half_h, t1,
                    x0 + dx0 * half_h,
                    dx0, x1 + dx1 * half_h, dx1); //top
        }
    }
}
```

Work and Cache Analysis

Recursion tree
for cache analysis



- The bottom of a leaf trapezoid just fits in the cache, so $w = \Theta(\mathcal{M})$.
- A leaf trapezoid contains $\Theta(hw) = \Theta(w^2)$ points and $\Theta(w^2)$ work.
- Since $w \leq \mathcal{M}$, a leaf incurs $\Theta(w/\mathcal{B})$ cache misses.
- There are $\Theta(NT/hw) = \Theta(NT/w^2)$ leaves and internal nodes.
- The internal nodes contribute little to both work and cache misses.
- Work = $\Theta(NT/w^2) \cdot \Theta(w^2) = \Theta(NT)$.
- Cache misses = $\Theta(NT/w^2) \cdot \Theta(w/\mathcal{B}) = \Theta(NT/\mathcal{B}w) = \Theta(NT/\mathcal{B}\mathcal{M})$.

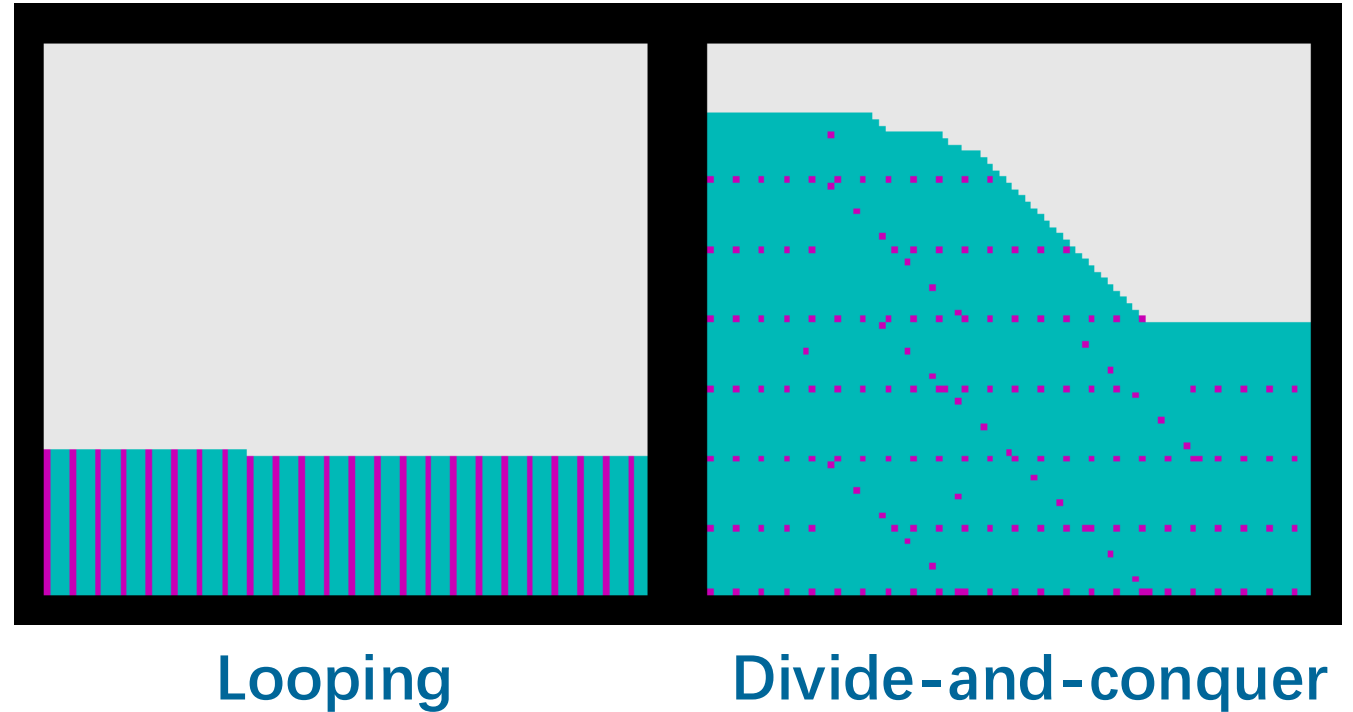
Simulation: 3-Point Stencil

- Rectangular region

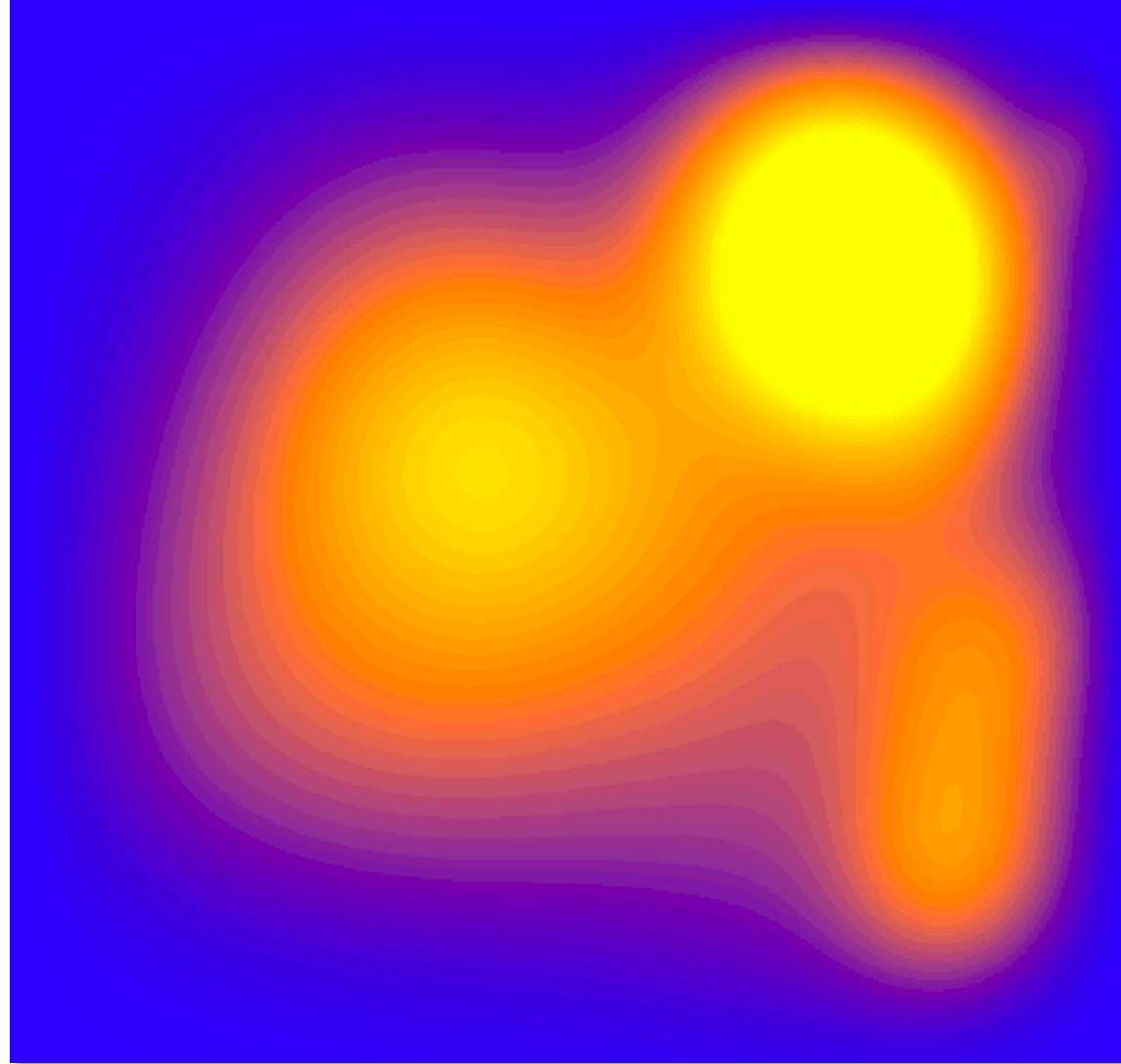
- ▣ $N = 95$

- ▣ $T = 87$

- Fully associative LRU cache
 - ▣ cache line $\mathcal{B} = 4$ points
 - ▣ cache size $\mathcal{M} = 32$ points = 8 cache lines
- Cache-hit latency = 1 cycle
- Cache-miss latency = 10 cycles



Looping vs. Trapezoid on Heat



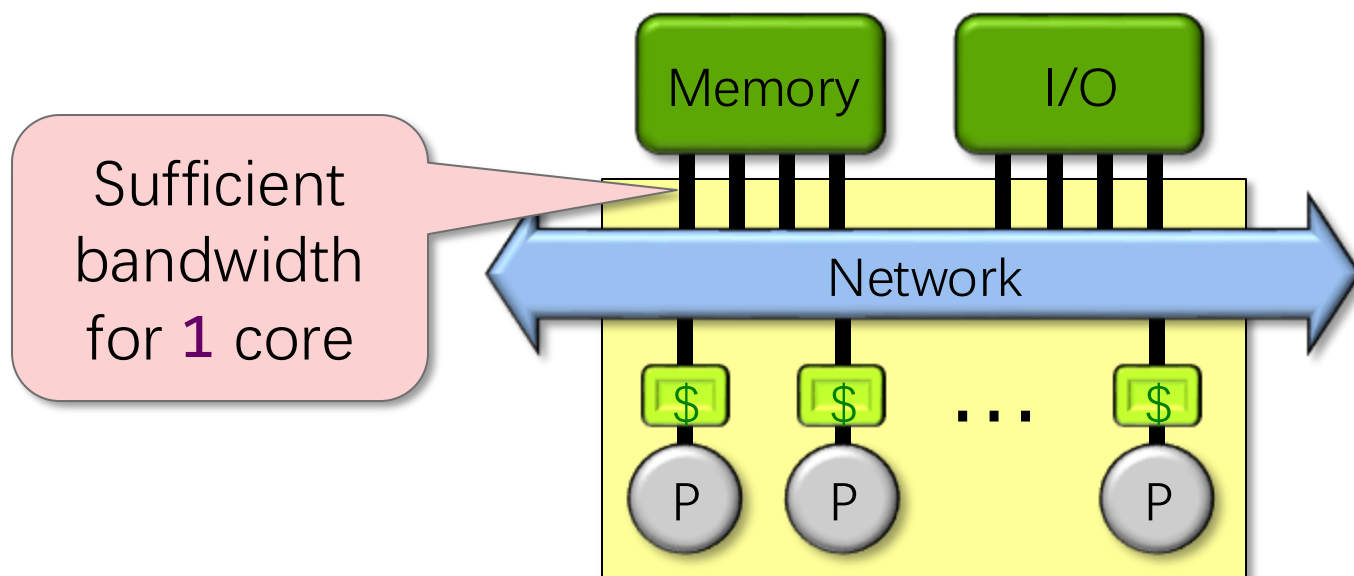
Impact on Performance

cache-oblivious algorithm incurs way fewer cache misses than **looping** version

Q. Why no performance gain?

A. **Prefetching** and a good memory architecture.

The memory bandwidth for one core largely suffices.

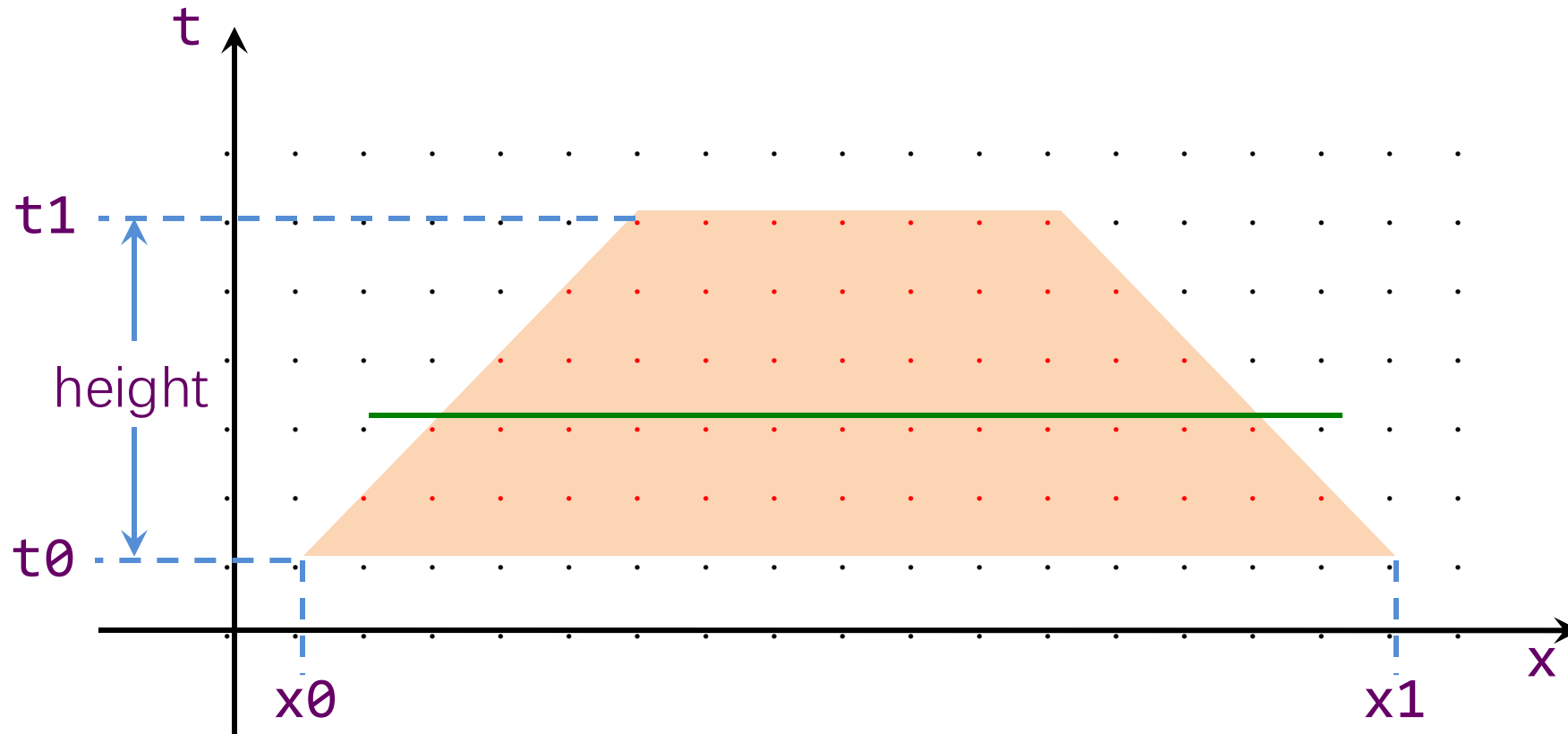


PARALLELIZING THE CACHE- OBLIVIOUS STENCIL COMPUTATION



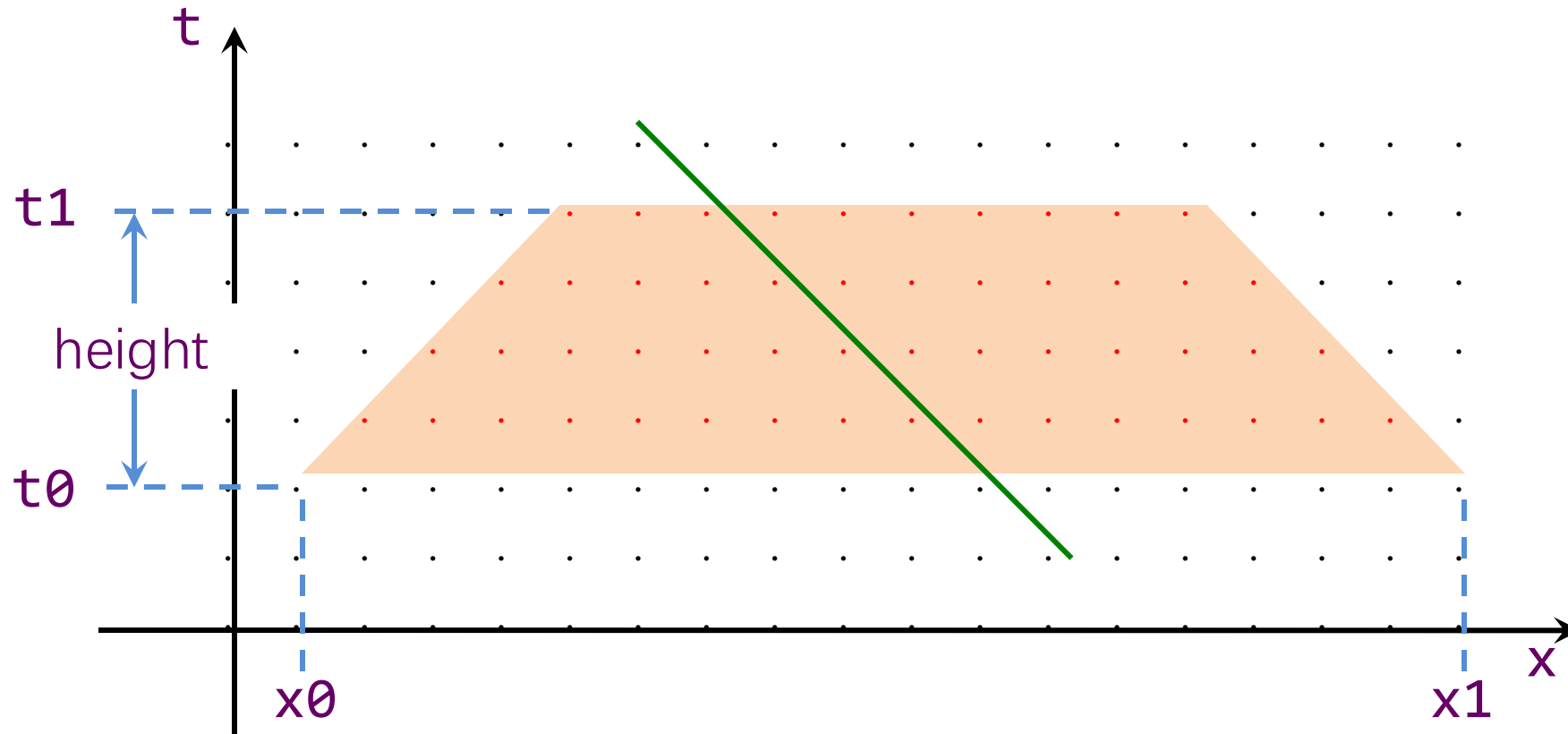
Time Cuts Don't Parallelize

- There's no way to parallelize a time cut
- The bottom trapezoid must be traversed first, and then the top one

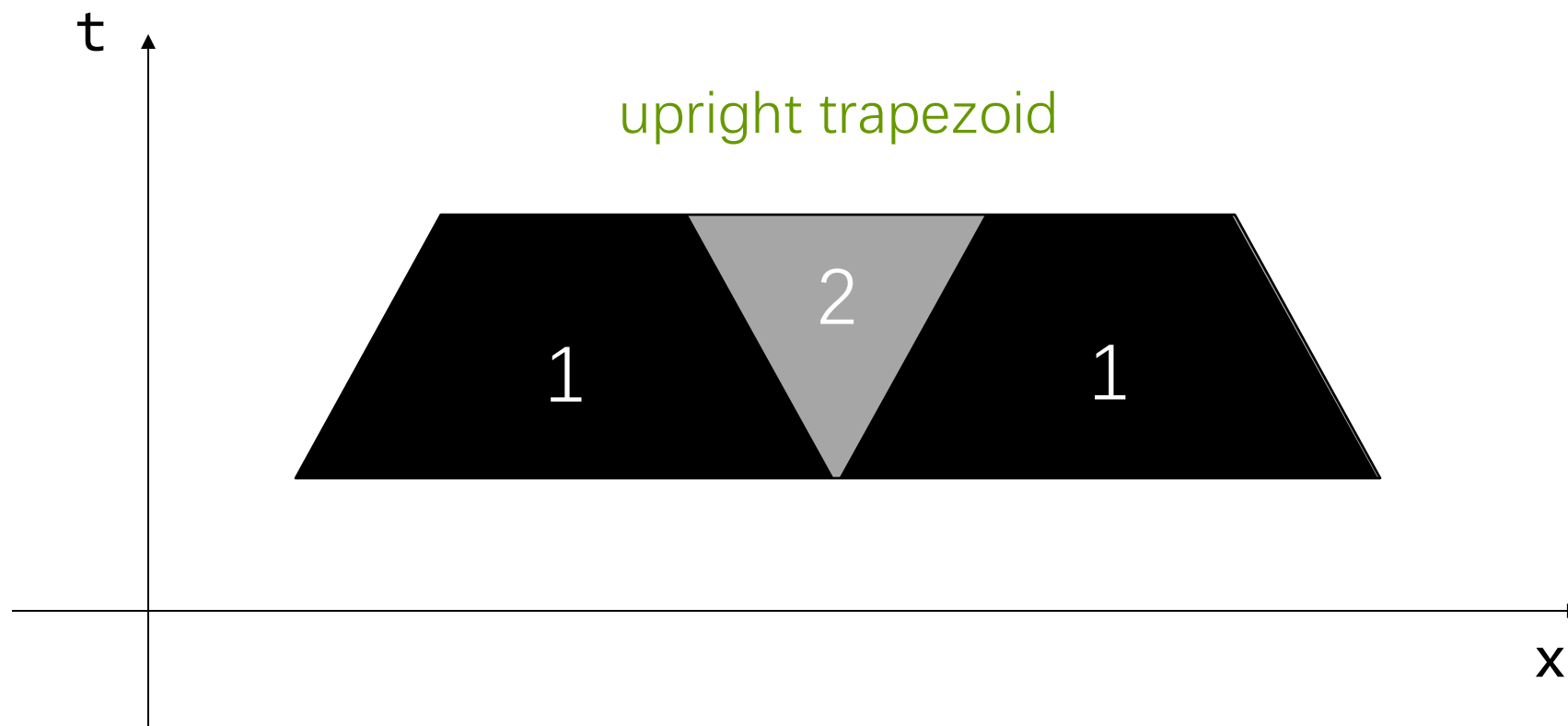


Space Cuts Don't Parallelize, or Do They?

- A space cut poses a similar problem
- Must traverse the trapezoid on the left before the one on the right

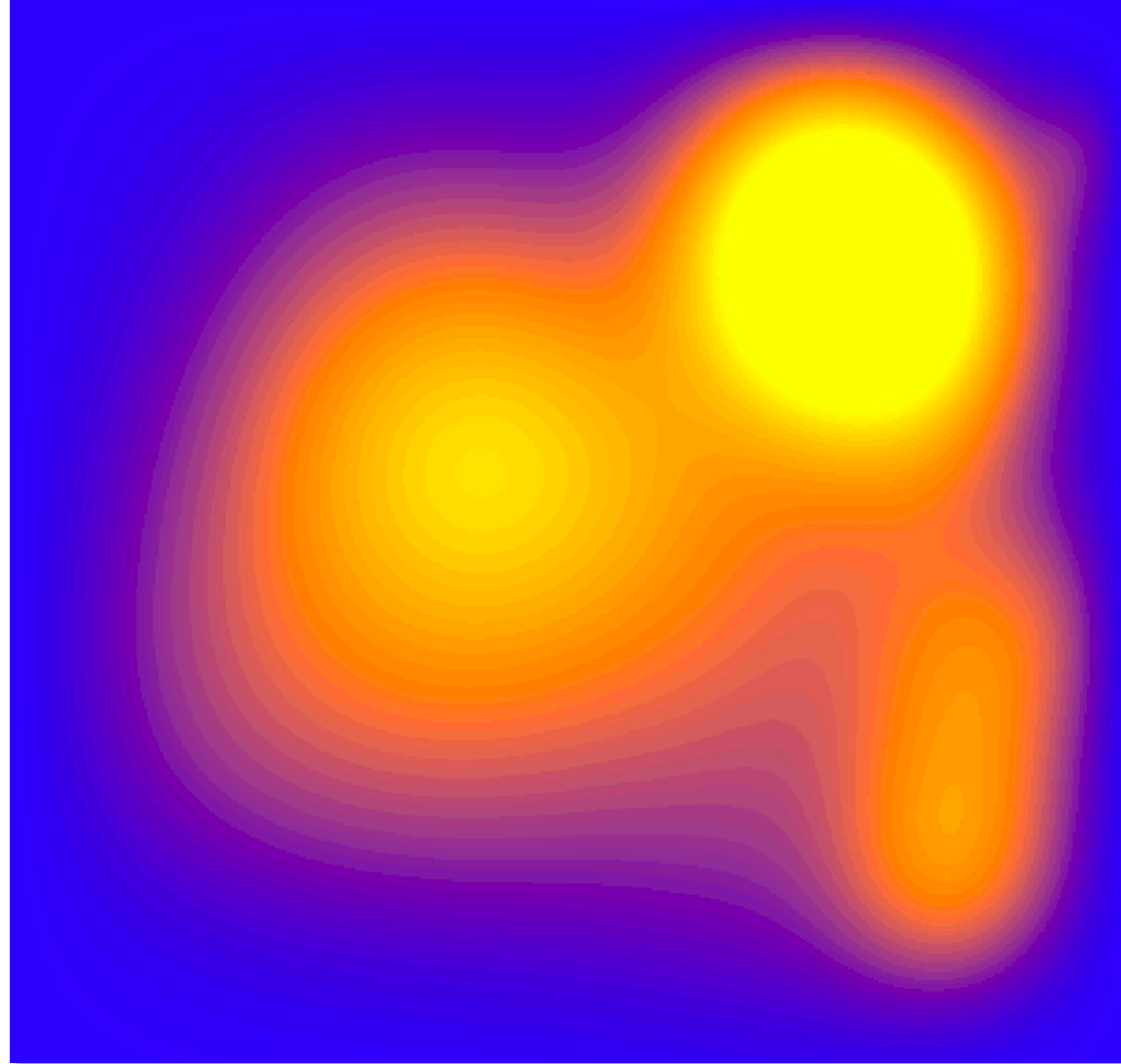


Parallel Space Cuts

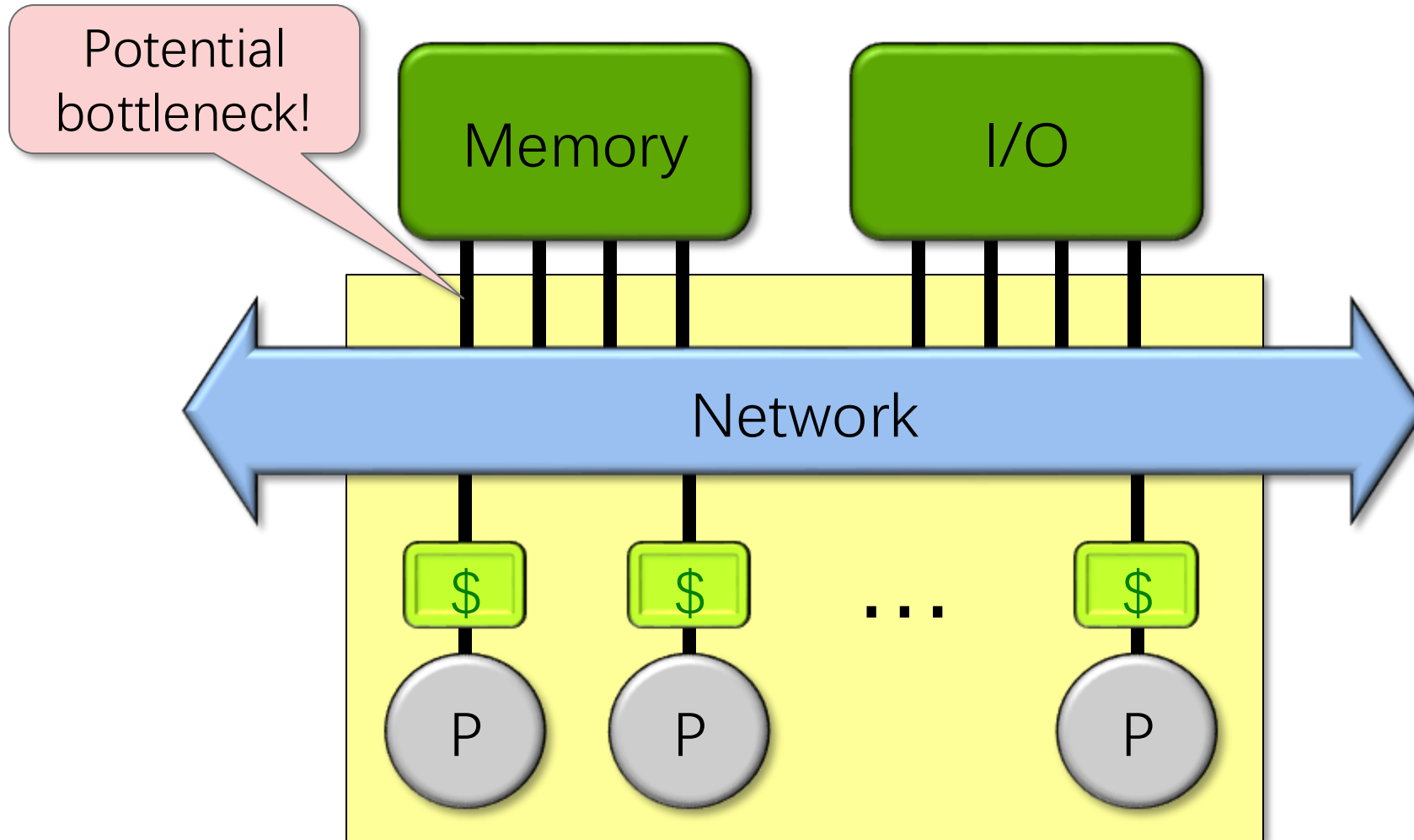


A **parallel space cut** produces two upright trapezoids (**black**) that can be executed in parallel and a third “inverted” trapezoid (**gray**) that must execute in series after the two upright trapezoids.

Parallel Looping vs. Parallel D&C



Memory Bandwidth



Impediments to Speedup

- ☒ Insufficient parallelism
- ☒ Scheduling overhead
- ☒ Lack of memory bandwidth
- ☐ Contention (locking and true/false sharing)

Cilkscale can diagnose the first two problems.

Q. How can we diagnose lack of memory bandwidth?

Impediments to Speedup

- ☒ Insufficient parallelism
- ☒ Scheduling overhead
- ☒ Lack of memory bandwidth
- ☐ Contention (locking and true/false sharing)

Cilkscale can diagnose the first two problems.

Q. How can we diagnose lack of memory bandwidth?

A. Run **P** identical copies of the serial projection in parallel
— if you have enough memory.

Tools exist to detect lock contention in an execution, but not the potential for lock contention. Potential for true and false sharing is even harder to detect, although you shouldn't have true sharing if your code is free of determinacy races.

Other C-O Algorithms

Matrix Transposition/Addition

$$\Theta(1 + mn/\mathcal{B})$$

Straightforward recursive algorithm.

Strassen's Algorithm

$$\Theta(n + n^2/\mathcal{B} + n^{\lg 7}/\mathcal{B}\mathcal{M}^{(\lg 7)/2 - 1})$$

Straightforward recursive algorithm.

Fast Fourier Transform

$$\Theta(1 + (n/\mathcal{B})(1 + \log_{\mathcal{M}} n))$$

Variant of Cooley-Tukey [CT65] using cache-oblivious matrix transpose.

LUP-Decomposition

$$\Theta(1 + n^2/\mathcal{B} + n^3/\mathcal{B}\mathcal{M}^{1/2})$$

Recursive algorithm due to Sivan Toledo [T97].

C-O Data Structures

Ordered-File Maintenance

$$O(1 + (\lg^2 n)/\mathcal{B})$$

INSERT/DELETE anywhere in file while maintaining $O(1)$ -sized gaps. Amortized bound [BDFC00], later improved in [BCDFC02].

B-Trees

INSERT/DELETE:

$$O(1 + \log_{\mathcal{B}+1} n + (\lg^2 n)/\mathcal{B})$$

SEARCH:

$$O(1 + \log_{\mathcal{B}+1} n)$$

TRAVERSE:

$$O(1 + k/\mathcal{B})$$

Solution [BDFC00] with later simplifications [BDIW02], [BFJ02].

Priority Queues

$$O(1 + (1/\mathcal{B}) \log_{\mathcal{M}/\mathcal{B}}(n/\mathcal{B}))$$

Funnel-based solution [BF02]. General scheme based on buffer trees [ABDHMM02] supports INSERT/DELETE.



Take-Aways



- Cache-oblivious algorithms can use cache as well as cache-aware algorithms without resorting to voodoo parameters.
- Cache-efficiency matters more for parallel code.
- The base case of divide-and-conquer algorithms must be engineered to account for hardware acceleration (prefetching, etc.) along the fast-moving index of a multidimensional array.
- The advantage of divide-and-conquer cache-oblivious algorithms over looping algorithms diminishes with the number of dimensions

CACHE-OBLIVIOUS SORTING (OMITTED)

This unit on sorting was not covered in lecture, but it has been taught in the past. It contains several instructive examples.

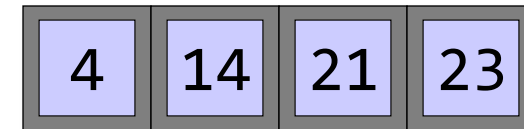
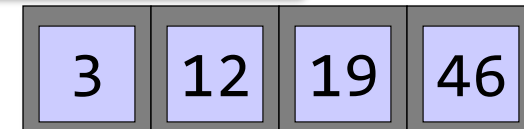


Merging Two Sorted Arrays

```
void merge(int64_t *C, int64_t *A, int64_t na,
           int64_t *B, int64_t nb) {
    while (na>0 && nb>0) {
        if (*A <= *B) {
            *C++ = *A++; na--;
        } else {
            *C++ = *B++; nb--;
        }
    }
    while (na>0) {
        *C++ = *A++; na--;
    }
    while (nb>0) {
        *C++ = *B++; nb--;
    }
}
```

Time to merge n
elements = $\Theta(n)$.

Number of cache
misses = $\Theta(n/B)$.



Merge Sort

```
void merge_sort(int64_t *B, int64_t *A, int64_t n) {  
    if (n==1) {  
        B[0] = A[0];  
    } else {  
        int64_t C[n];  
        cilk_spawn merge_sort(C, A, n/2);  
                   merge_sort(C+n/2, A+n/2, n-n/2);  
        cilk_sync;  
        merge(B, C, n/2, C+n/2, n-n/2);  
    }  
}
```

19 3 12 46 33 4 21 14

Merge Sort

```
void merge_sort(int64_t *B, int64_t *A, int64_t n) {  
    if (n==1) {  
        B[0] = A[0];  
    } else {  
        int64_t C[n];  
        cilk_spawn merge_sort(C, A, n/2);  
                   merge_sort(C+n/2, A+n/2, n-n/2);  
        cilk_sync;  
        merge(B, C, n/2, C+n/2, n-n/2);  
    }  
}
```

19 3 12 46

33 4 21 14

Merge Sort

```
void merge_sort(int64_t *B, int64_t *A, int64_t n) {  
    if (n==1) {  
        B[0] = A[0];  
    } else {  
        int64_t C[n];  
        cilk_spawn merge_sort(C, A, n/2);  
                   merge_sort(C+n/2, A+n/2, n-n/2);  
        cilk_sync;  
        merge(B, C, n/2, C+n/2, n-n/2);  
    }  
}
```

19 3

12 46

33 4

21 14

Merge Sort

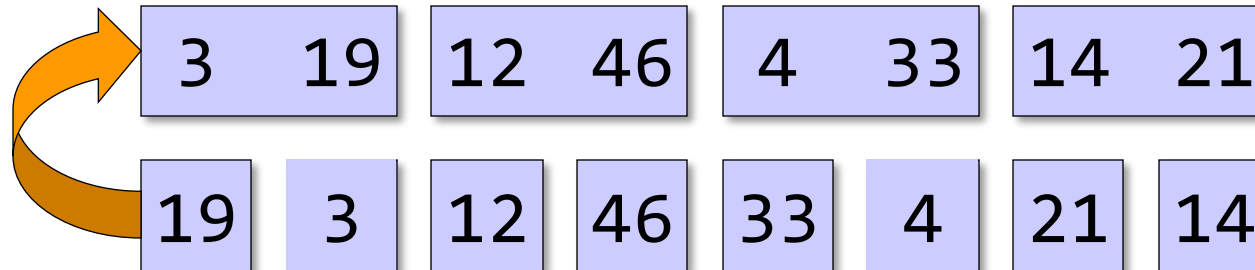
```
void merge_sort(int64_t *B, int64_t *A, int64_t n) {  
    if (n==1) {  
        B[0] = A[0];  
    } else {  
        int64_t C[n];  
        cilk_spawn merge_sort(C, A, n/2);  
                   merge_sort(C+n/2, A+n/2, n-n/2);  
        cilk_sync;  
        merge(B, C, n/2, C+n/2, n-n/2);  
    }  
}
```

19	3	12	46	33	4	21	14
----	---	----	----	----	---	----	----

Merge Sort

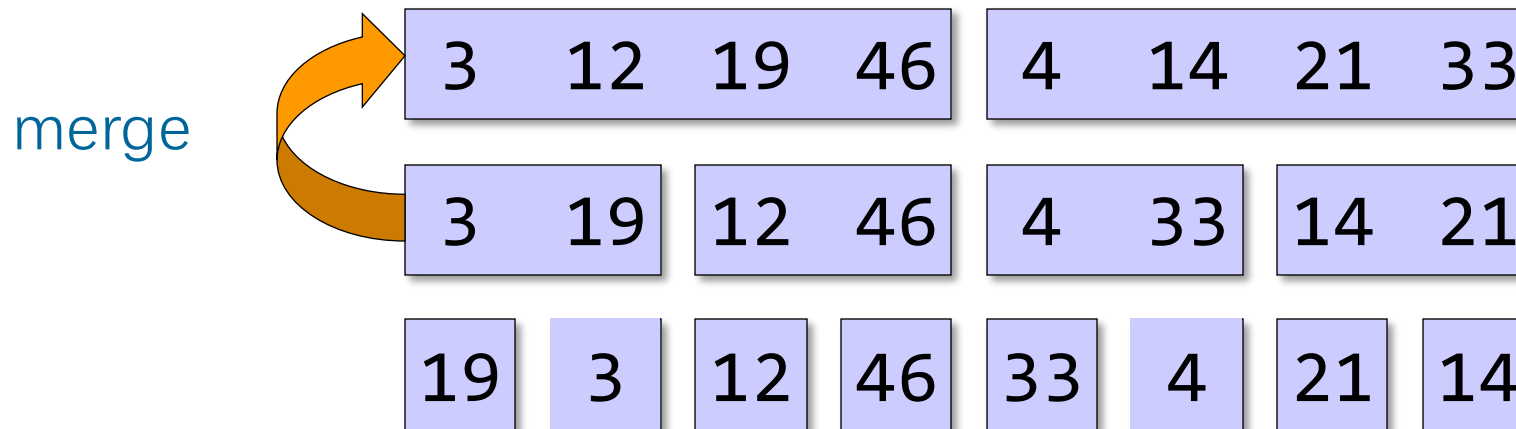
```
void merge_sort(int64_t *B, int64_t *A, int64_t n) {  
    if (n==1) {  
        B[0] = A[0];  
    } else {  
        int64_t C[n];  
        cilk_spawn merge_sort(C, A, n/2);  
                   merge_sort(C+n/2, A+n/2, n-n/2);  
        cilk_sync;  
        merge(B, C, n/2, C+n/2, n-n/2);  
    }  
}
```

merge



Merge Sort

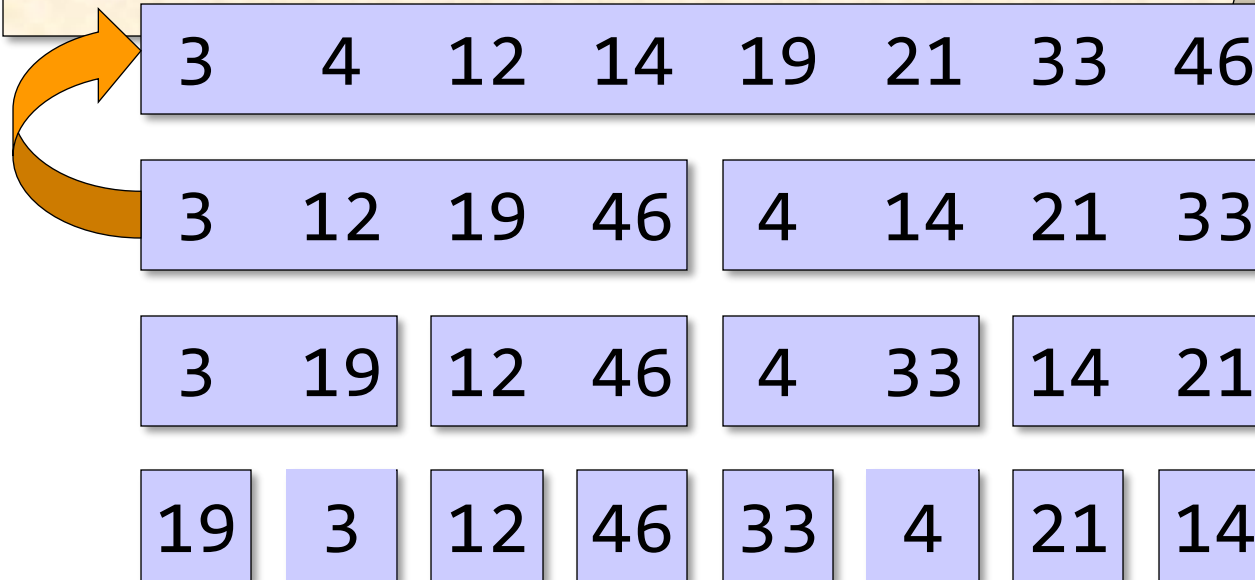
```
void merge_sort(int64_t *B, int64_t *A, int64_t n) {  
    if (n==1) {  
        B[0] = A[0];  
    } else {  
        int64_t C[n];  
        cilk_spawn merge_sort(C, A, n/2);  
                   merge_sort(C+n/2, A+n/2, n-n/2);  
        cilk_sync;  
        merge(B, C, n/2, C+n/2, n-n/2);  
    }  
}
```



Merge Sort

```
void merge_sort(int64_t *B, int64_t *A, int64_t n) {  
    if (n==1) {  
        B[0] = A[0];  
    } else {  
        int64_t C[n];  
        cilk_spawn merge_sort(C, A, n/2);  
                   merge_sort(C+n/2, A+n/2, n-n/2);  
        cilk_sync;  
        merge(B, C, n/2, C+n/2, n-n/2);  
    }  
}
```

merge



Work of Merge Sort

```
void merge_sort(int64_t *B, int64_t *A, int64_t n) {  
    if (n==1) {  
        B[0] = A[0];  
    } else {  
        int64_t C[n];  
        merge_sort(C, A, n/2);  
        merge_sort(C+n/2, A+n/2, n-n/2);  
        merge(B, C, n/2, C+n/2, n-n/2);  
    }  
}
```

CASE 2

$$n^{\log_b a} = n^{\log_2 2} = n$$

$$f(n) = \Theta(n^{\log_b a} \lg^0 n)$$

Work:
$$W(n) = 2W(n/2) + \Theta(n)$$
$$= \Theta(n \lg n)$$

Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.

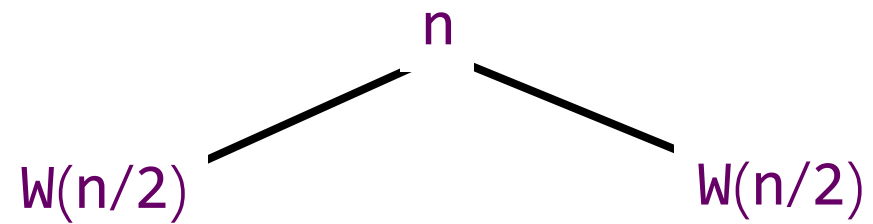
Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.

$W(n)$

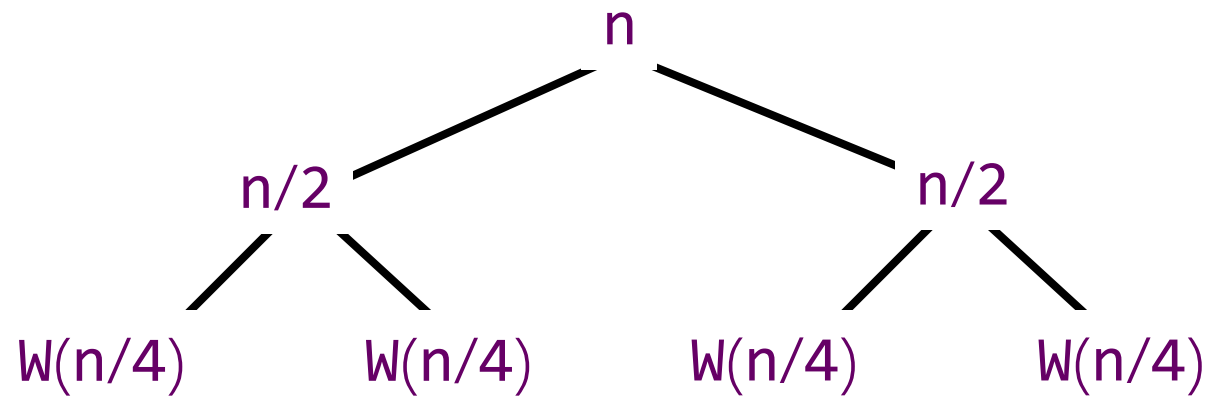
Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.



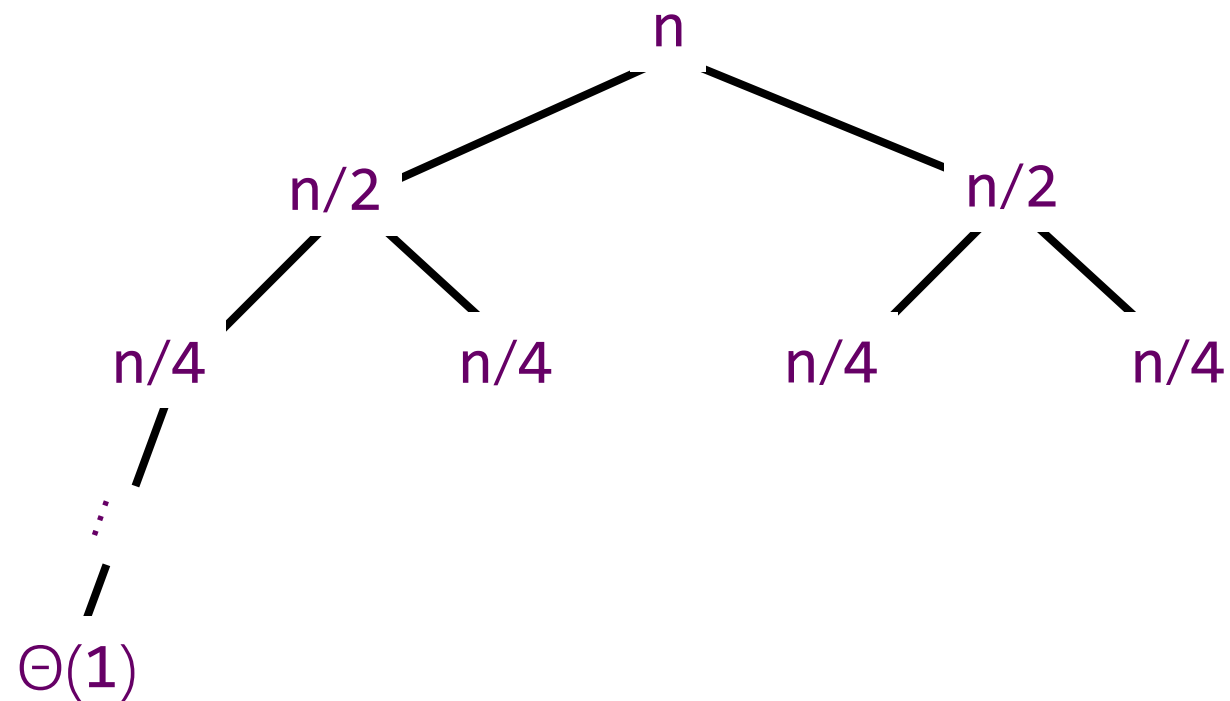
Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.



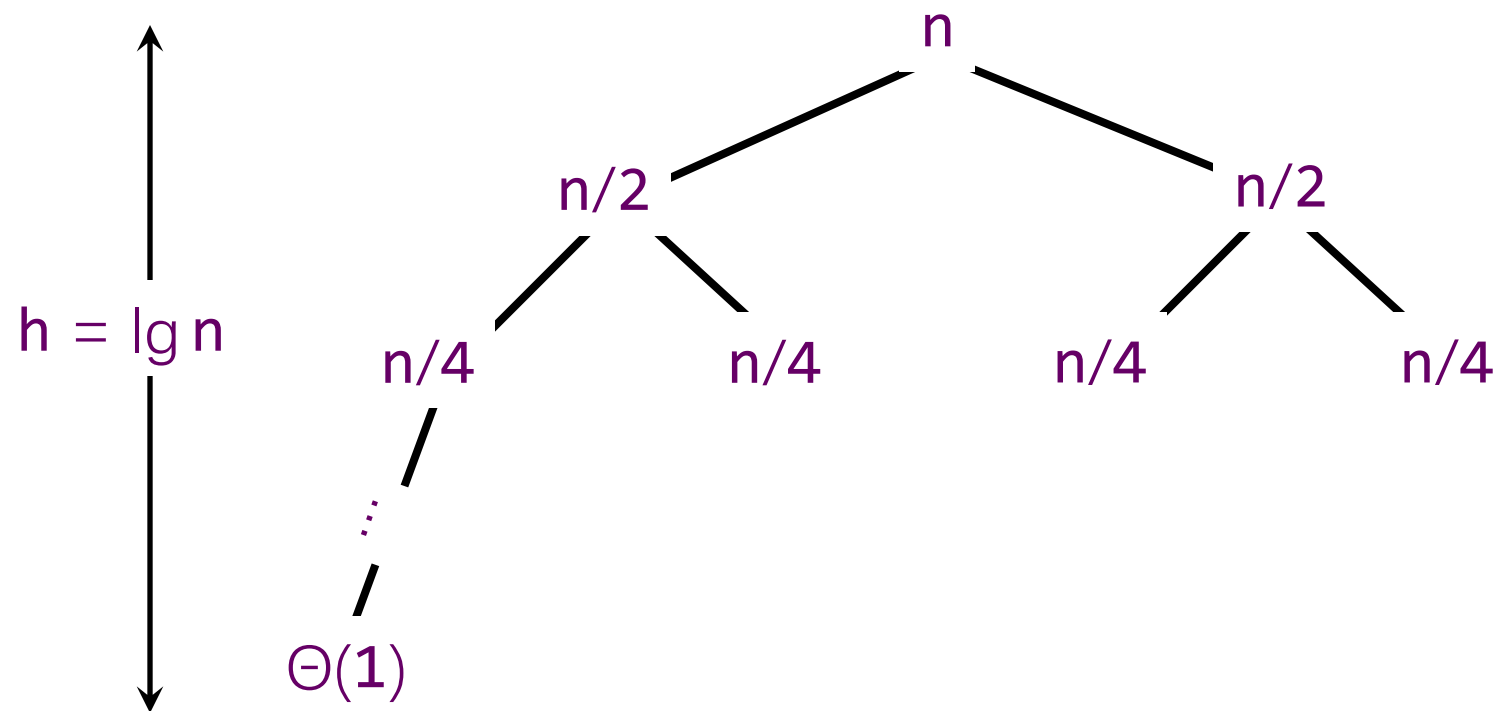
Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.



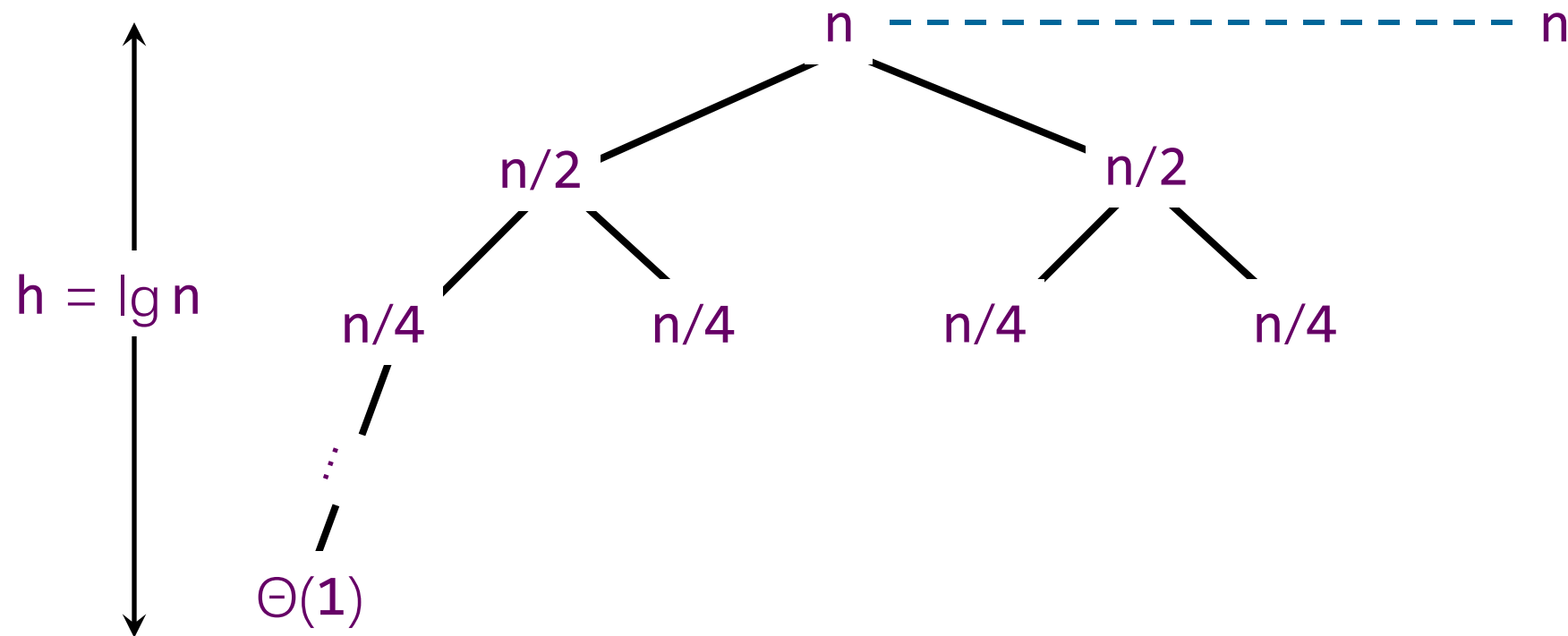
Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.



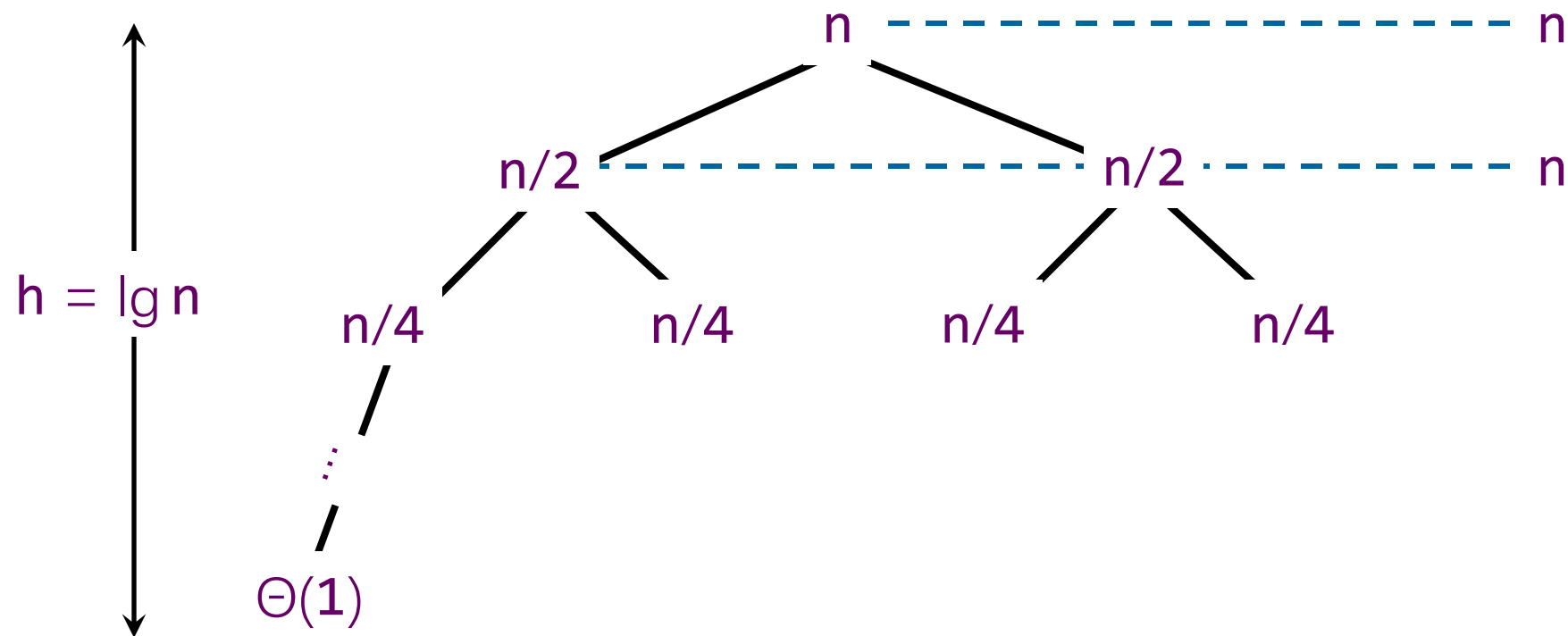
Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.



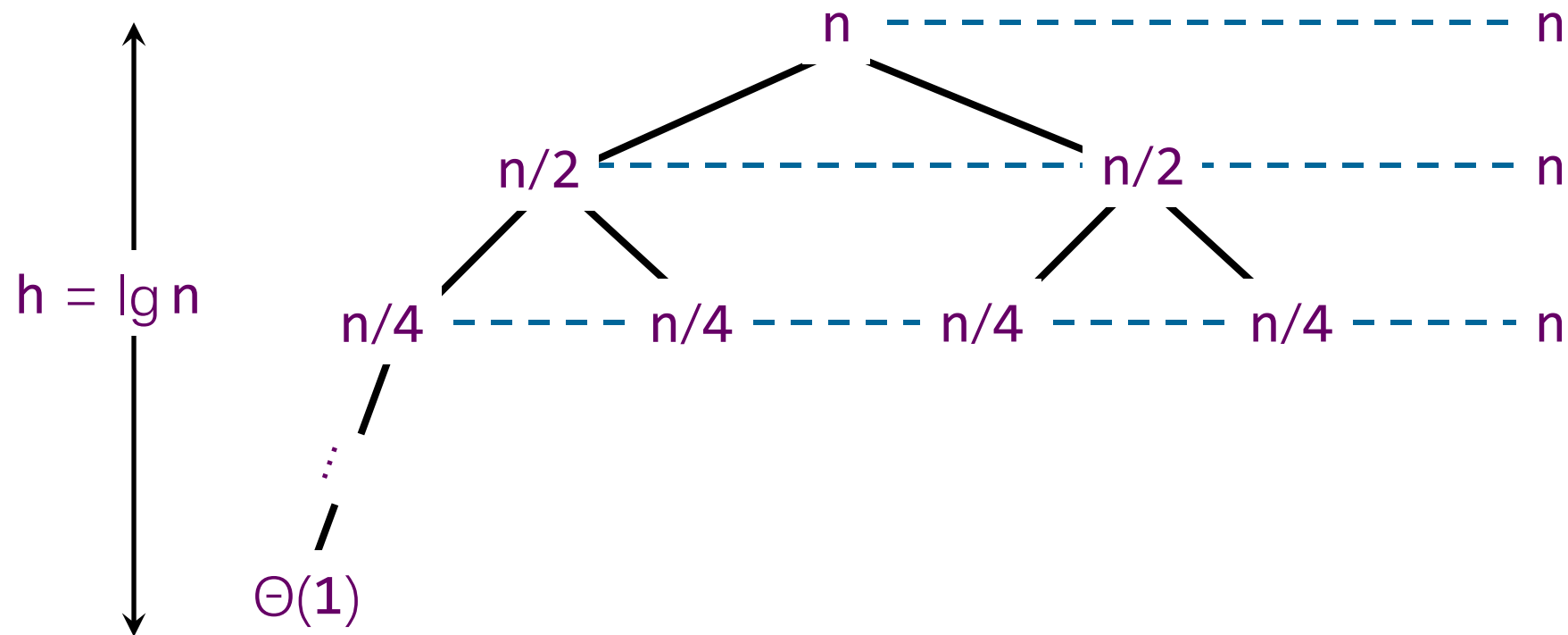
Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.



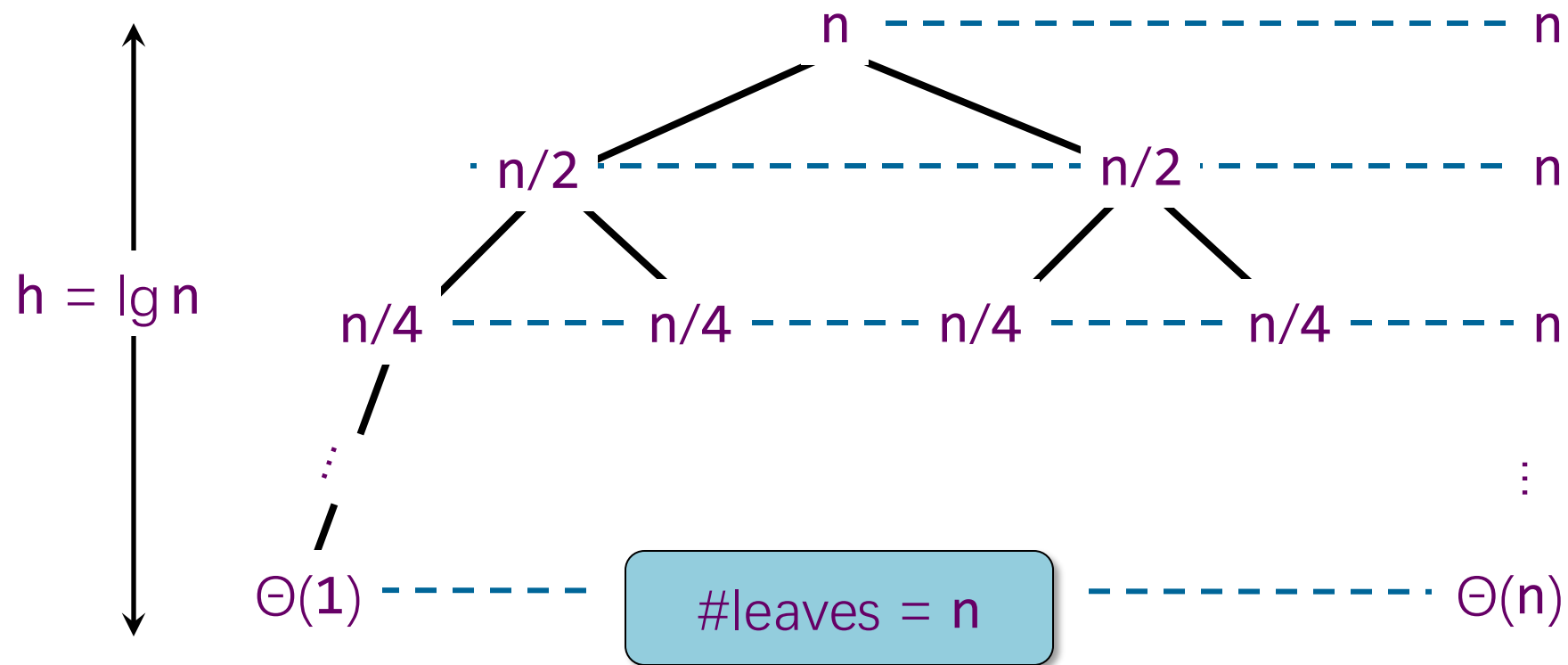
Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.



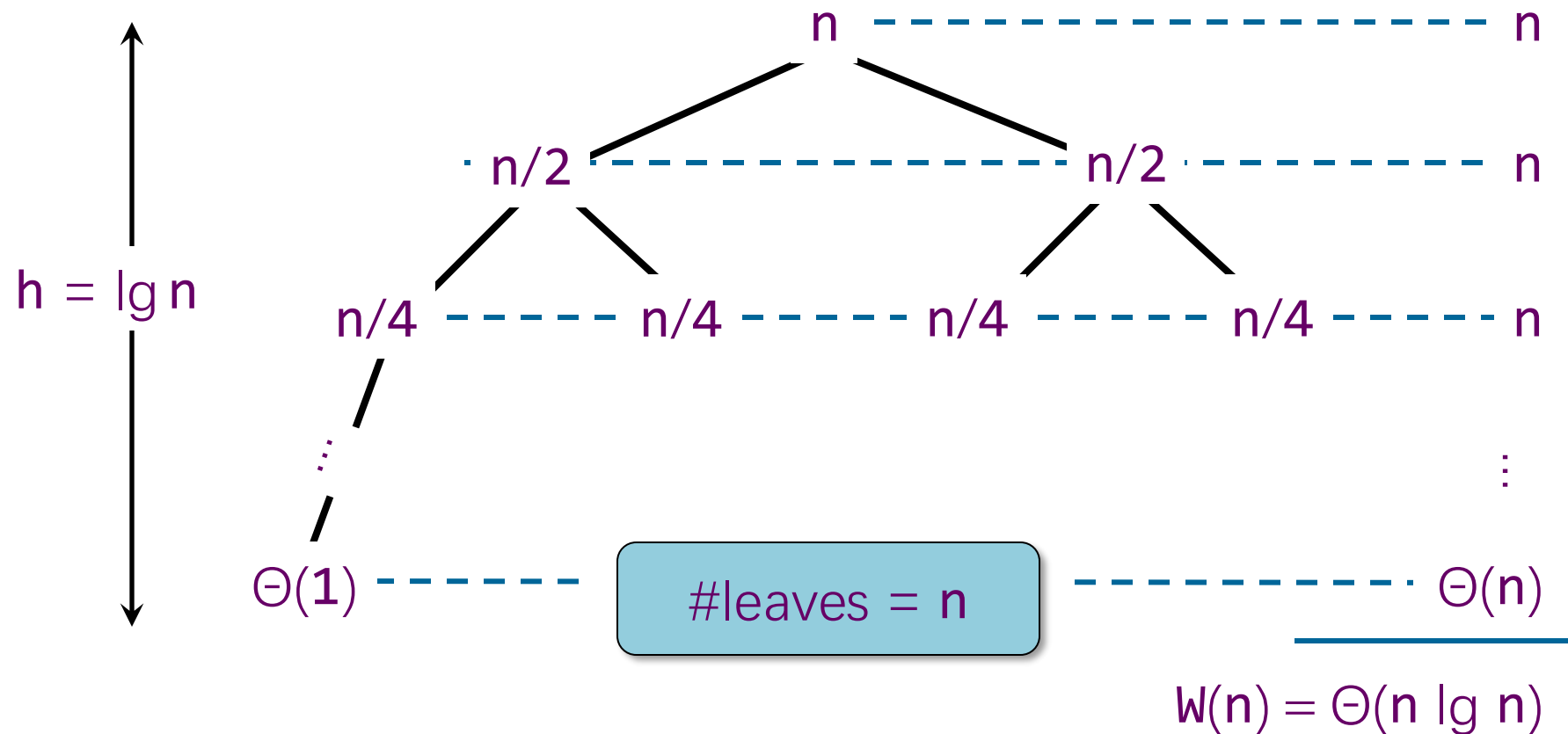
Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.



Recursion Tree

Solve $W(n) = 2W(n/2) + \Theta(n)$.



Now with Caching

Merge subroutine

$$Q(n) = \Theta(n/\mathcal{B}) .$$

Merge sort

$$Q(n) = \begin{cases} \Theta(n/\mathcal{B}) & \text{if } n \leq c\mathcal{M}, \text{ constant } c \leq 1; \\ 2Q(n/2) + \Theta(n/\mathcal{B}) & \text{otherwise.} \end{cases}$$

Cache Analysis of Merge Sort

$$Q(n) = \begin{cases} \Theta(n/\mathcal{B}) & \text{if } n \leq c\mathcal{M}, \text{ constant } c \leq 1; \\ 2Q(n/2) + \Theta(n/\mathcal{B}) & \text{otherwise.} \end{cases}$$

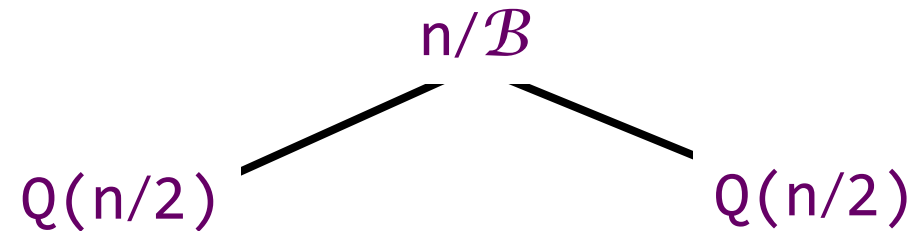
Recursion tree

$Q(n)$

Cache Analysis of Merge Sort

$$Q(n) = \begin{cases} \Theta(n/\mathcal{B}) & \text{if } n \leq c\mathcal{M}, \text{ constant } c \leq 1; \\ 2Q(n/2) + \Theta(n/\mathcal{B}) & \text{otherwise.} \end{cases}$$

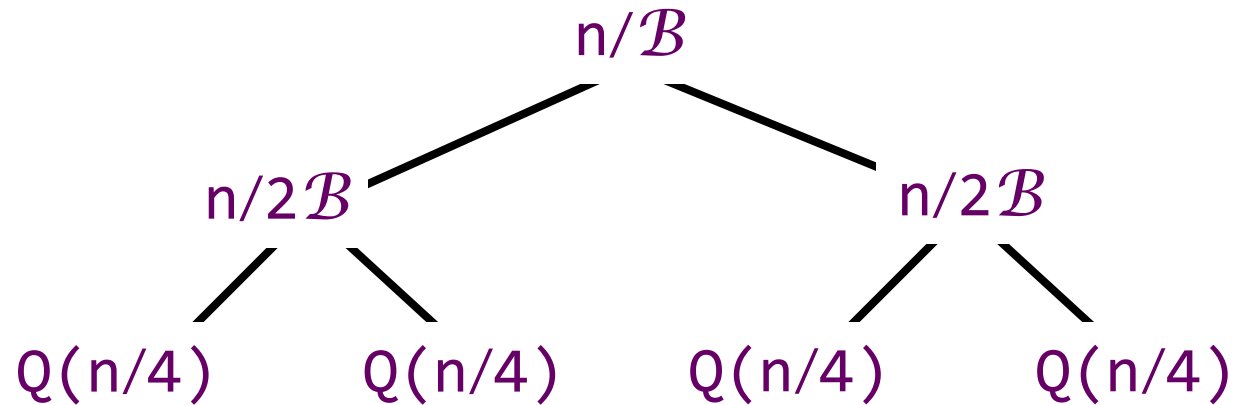
Recursion tree



Cache Analysis of Merge Sort

$$Q(n) = \begin{cases} \Theta(n/\mathcal{B}) & \text{if } n \leq c\mathcal{M}, \text{ constant } c \leq 1; \\ 2Q(n/2) + \Theta(n/\mathcal{B}) & \text{otherwise.} \end{cases}$$

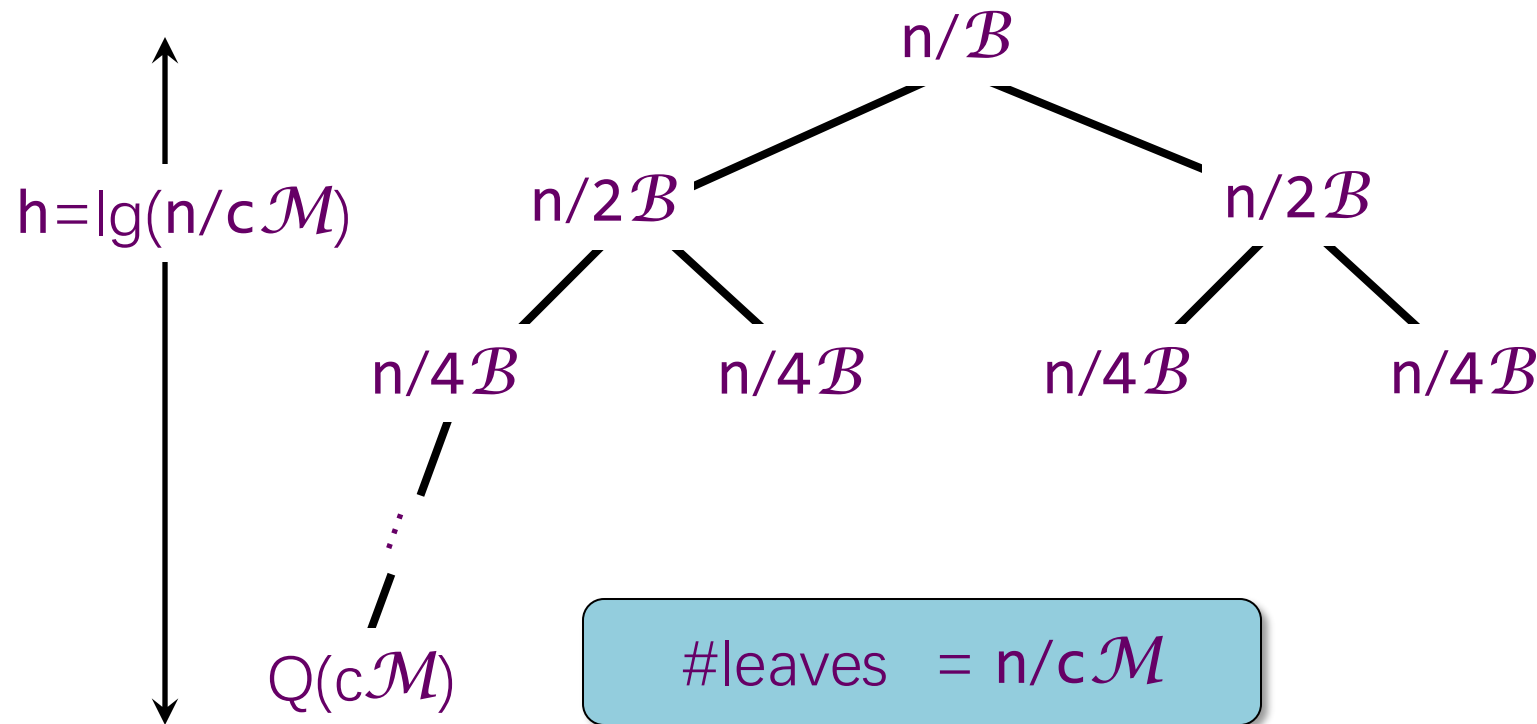
Recursion tree



Cache Analysis of Merge Sort

$$Q(n) = \begin{cases} \Theta(n/\mathcal{B}) & \text{if } n \leq c\mathcal{M}, \text{ constant } c \leq 1; \\ 2Q(n/2) + \Theta(n/\mathcal{B}) & \text{otherwise.} \end{cases}$$

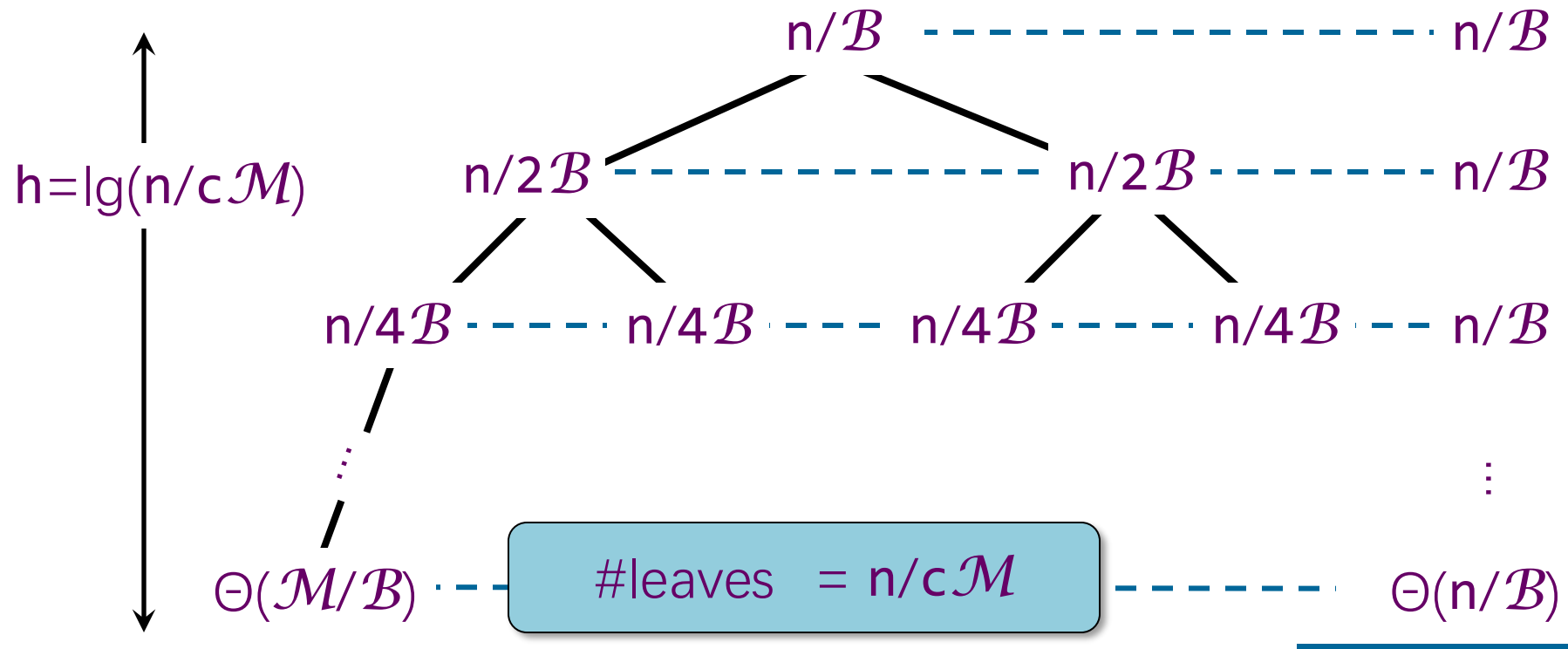
Recursion tree



Cache Analysis of Merge Sort

$$Q(n) = \begin{cases} \Theta(n/\mathcal{B}) & \text{if } n \leq c\mathcal{M}, \text{ constant } c \leq 1; \\ 2Q(n/2) + \Theta(n/\mathcal{B}) & \text{otherwise.} \end{cases}$$

Recursion tree



$$Q(n) = \Theta((n/\mathcal{B}) \lg(n/\mathcal{M}))$$

Bottom Line for Merge Sort

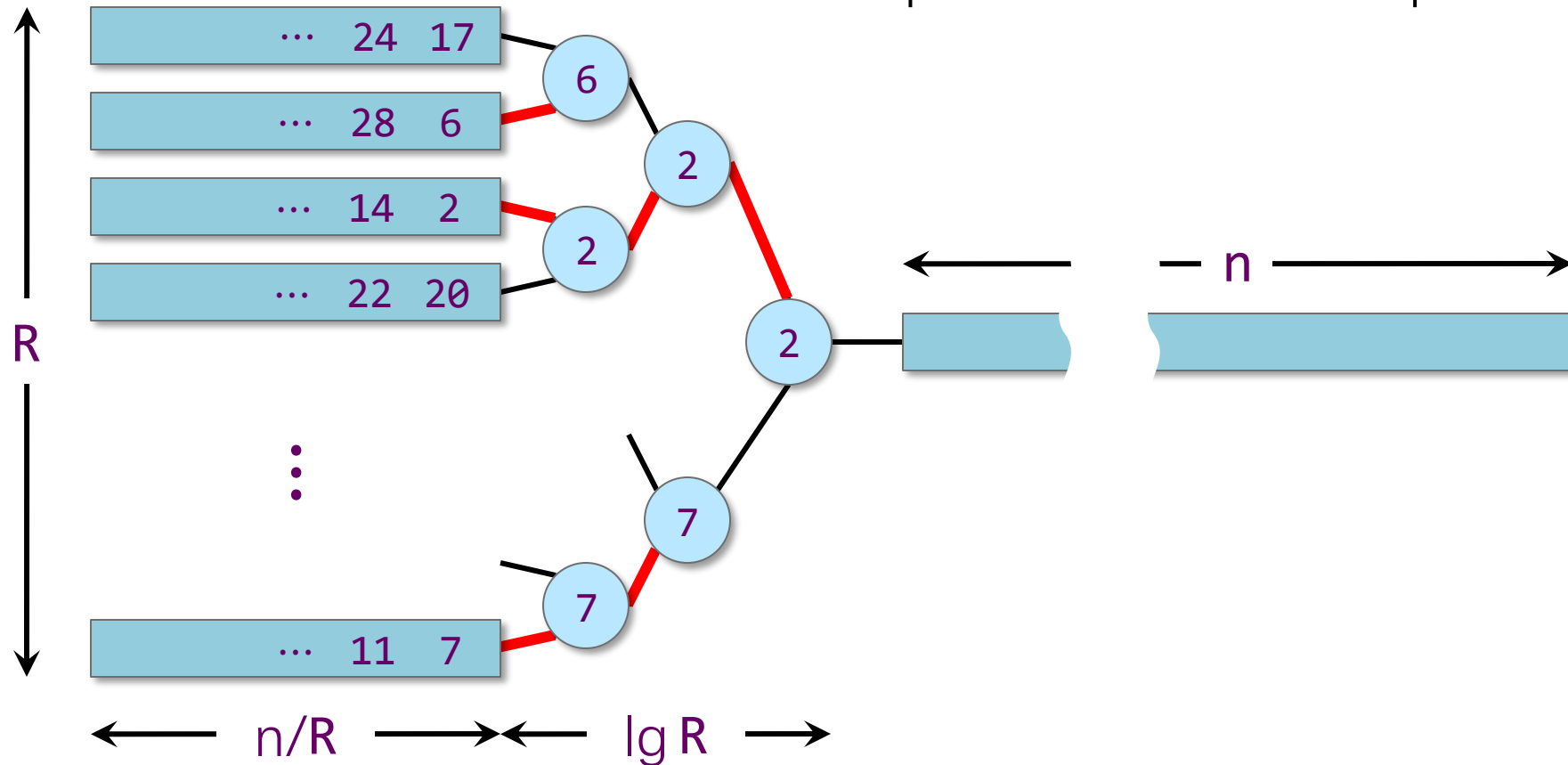
$$\begin{aligned} Q(n) &= \begin{cases} \Theta(n/\mathcal{B}) & \text{if } n \leq c\mathcal{M}, \text{ constant } c \leq 1; \\ 2Q(n/2) + \Theta(n/\mathcal{B}) & \text{otherwise;} \end{cases} \\ &= \Theta((n/\mathcal{B}) \lg(n/\mathcal{M})). \end{aligned}$$

- For $n \gg \mathcal{M}$, we have $\lg(n/\mathcal{M}) \approx \lg n$, and thus $W(n)/Q(n) \approx \Theta(\mathcal{B})$.
- For $n \approx \mathcal{M}$, we have $\lg(n/\mathcal{M}) \approx \Theta(1)$, and thus $W(n)/Q(n) \approx \Theta(\mathcal{B} \lg n)$.

Multiway Merging

IDEA: Merge $R < n$ subarrays with a tournament.

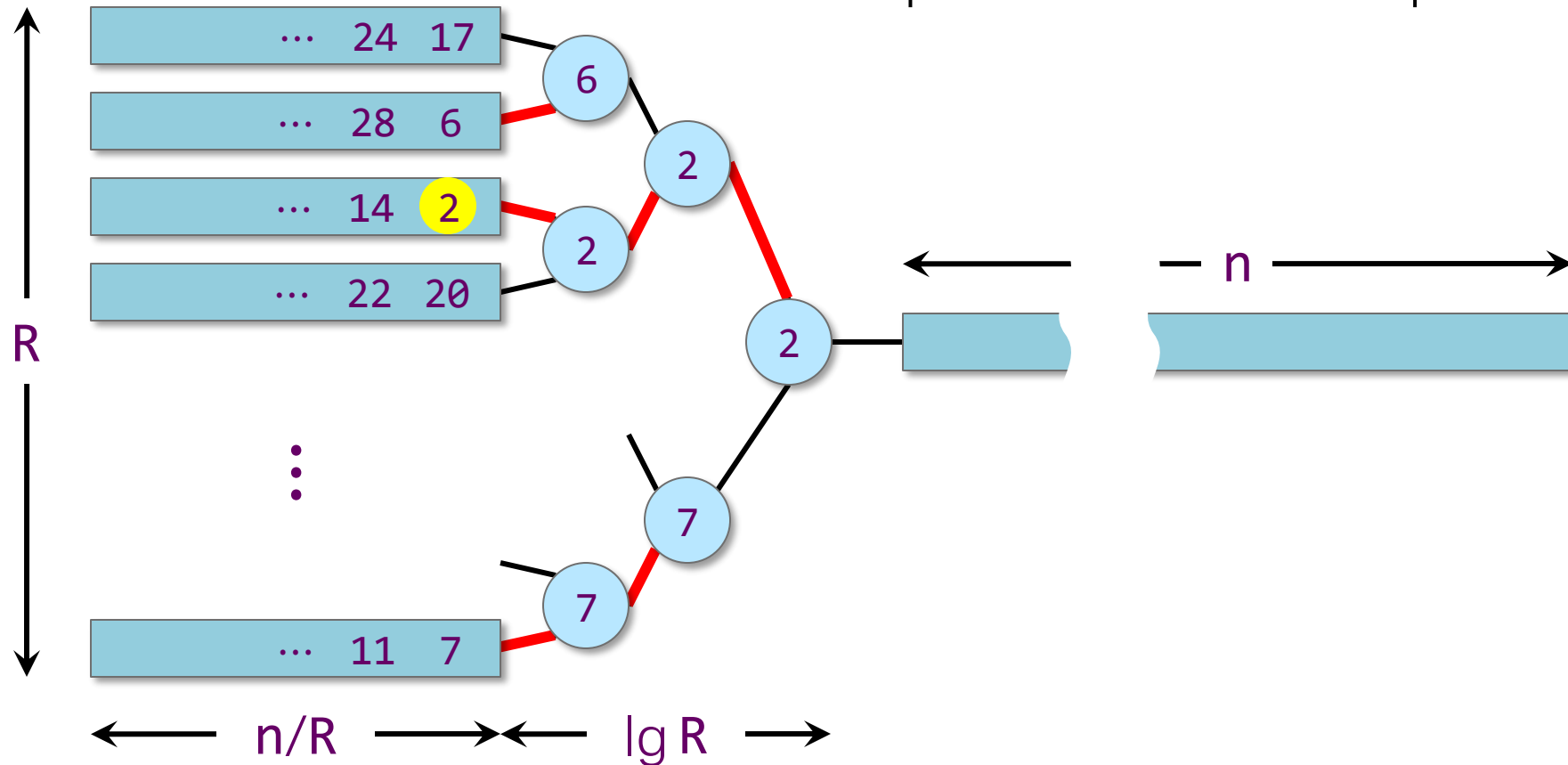
- Tournament takes $\Theta(R)$ work to produce the first output.



Multiway Merging

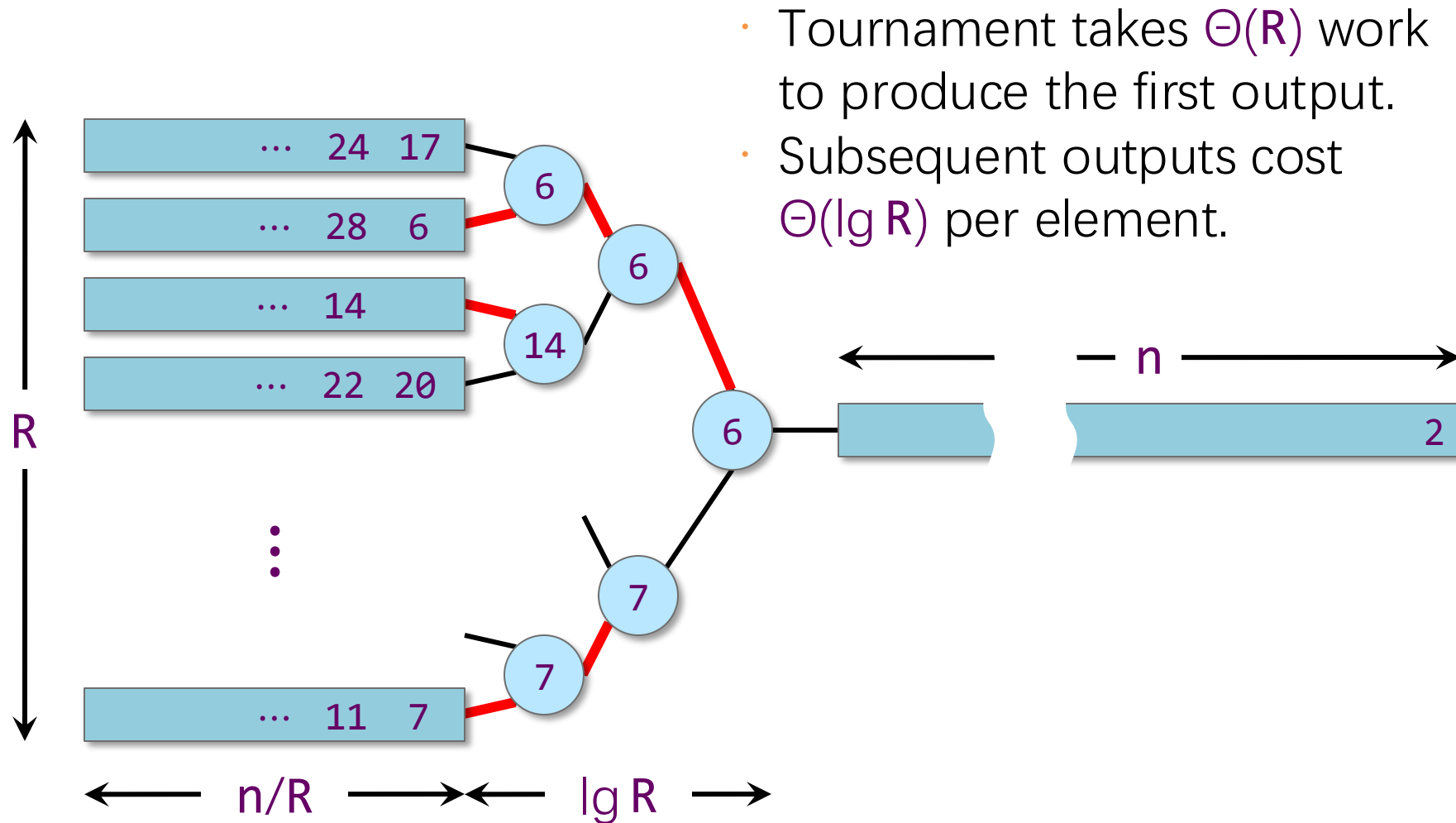
IDEA: Merge $R < n$ subarrays with a tournament.

- Tournament takes $\Theta(R)$ work to produce the first output.



Multiway Merging

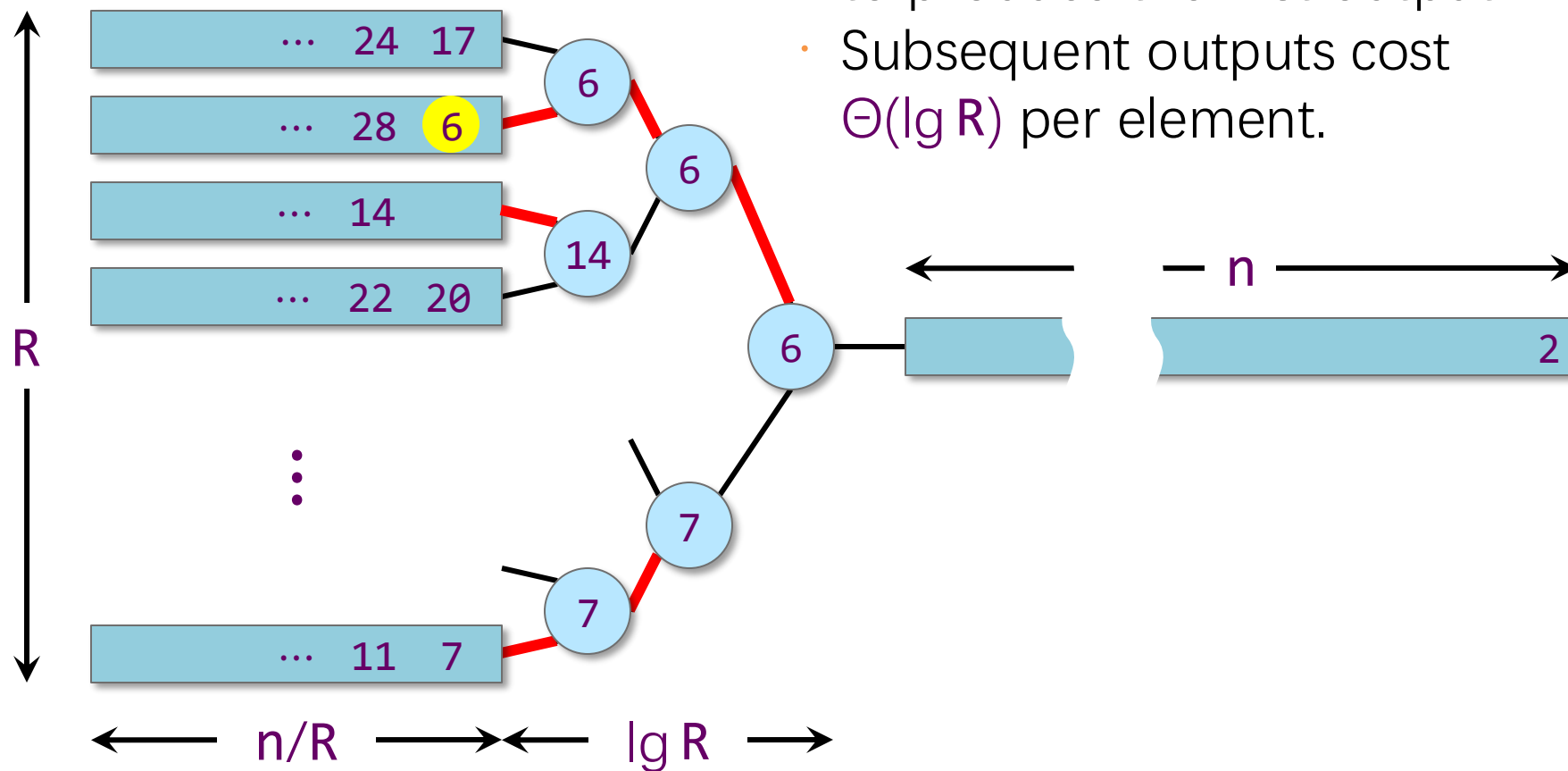
IDEA: Merge $R < n$ subarrays with a tournament.



Multiway Merging

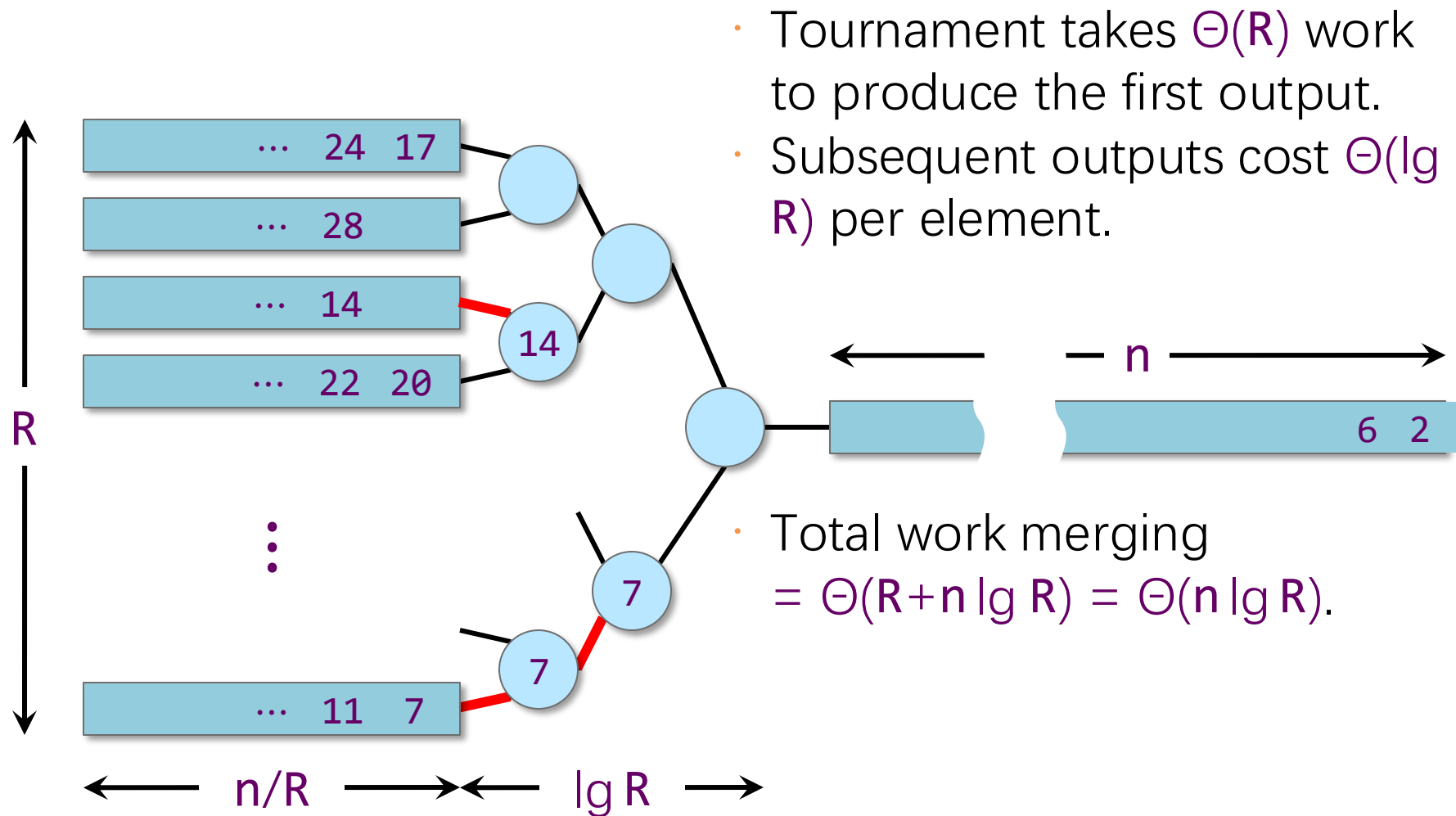
IDEA: Merge $R < n$ subarrays with a tournament.

- Tournament takes $\Theta(R)$ work to produce the first output.
- Subsequent outputs cost $\Theta(\lg R)$ per element.



Multiway Merging

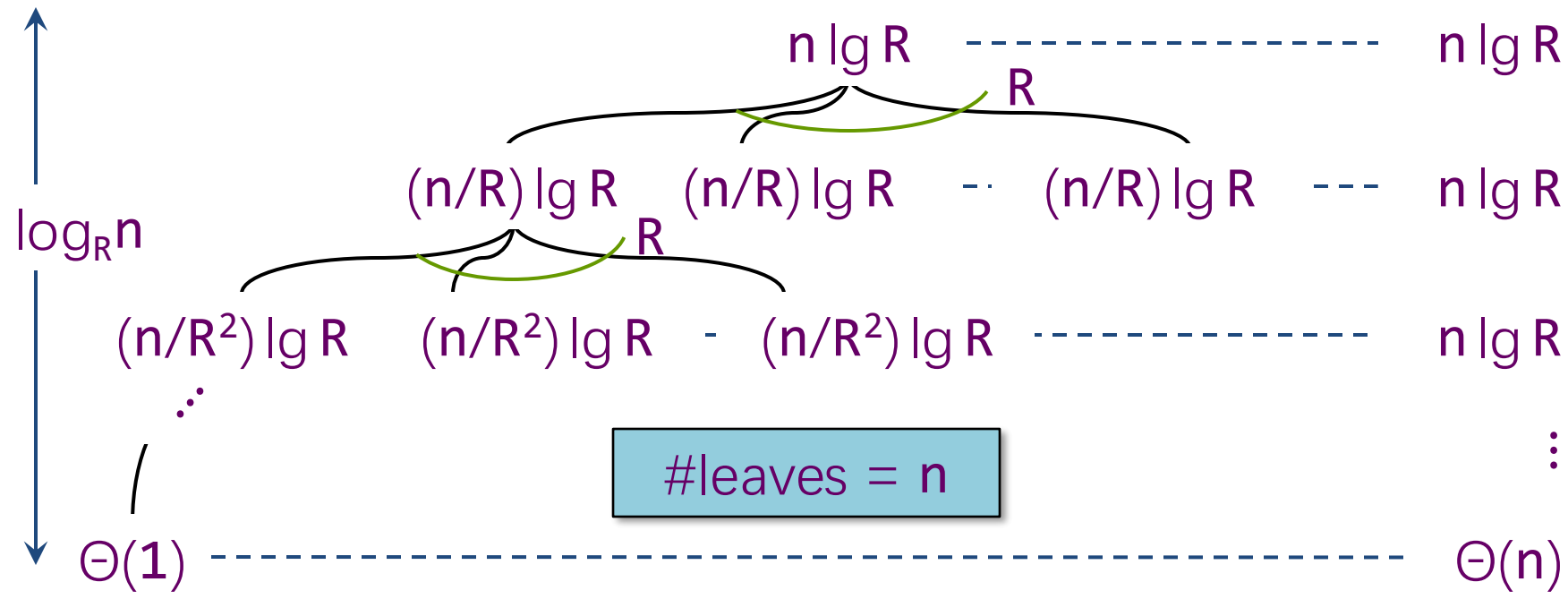
IDEA: Merge $R < n$ subarrays with a tournament.



Work of Multiway Merge Sort

$$W(n) = \begin{cases} \Theta(1) & \text{if } n = 1; \\ R \cdot W(n/R) + \Theta(n \lg R) & \text{otherwise.} \end{cases}$$

Recursion tree



Same as binary merge sort.

$$\begin{aligned} W(n) &= \Theta((n \lg R) \log_R n + n) \\ &= \Theta((n \lg R)(\lg n) / \lg R + n) \\ &= \Theta(n \lg n) \end{aligned}$$

Caching Recurrence

Consider the R -way merging of contiguous arrays of total size n . If $R < c\mathcal{M}/\mathcal{B}$, for some sufficiently small constant $c \leq 1$, the entire tournament plus 1 block from each array can fit in cache.

$\Rightarrow Q(n) \leq \Theta(n/\mathcal{B})$ for merging.

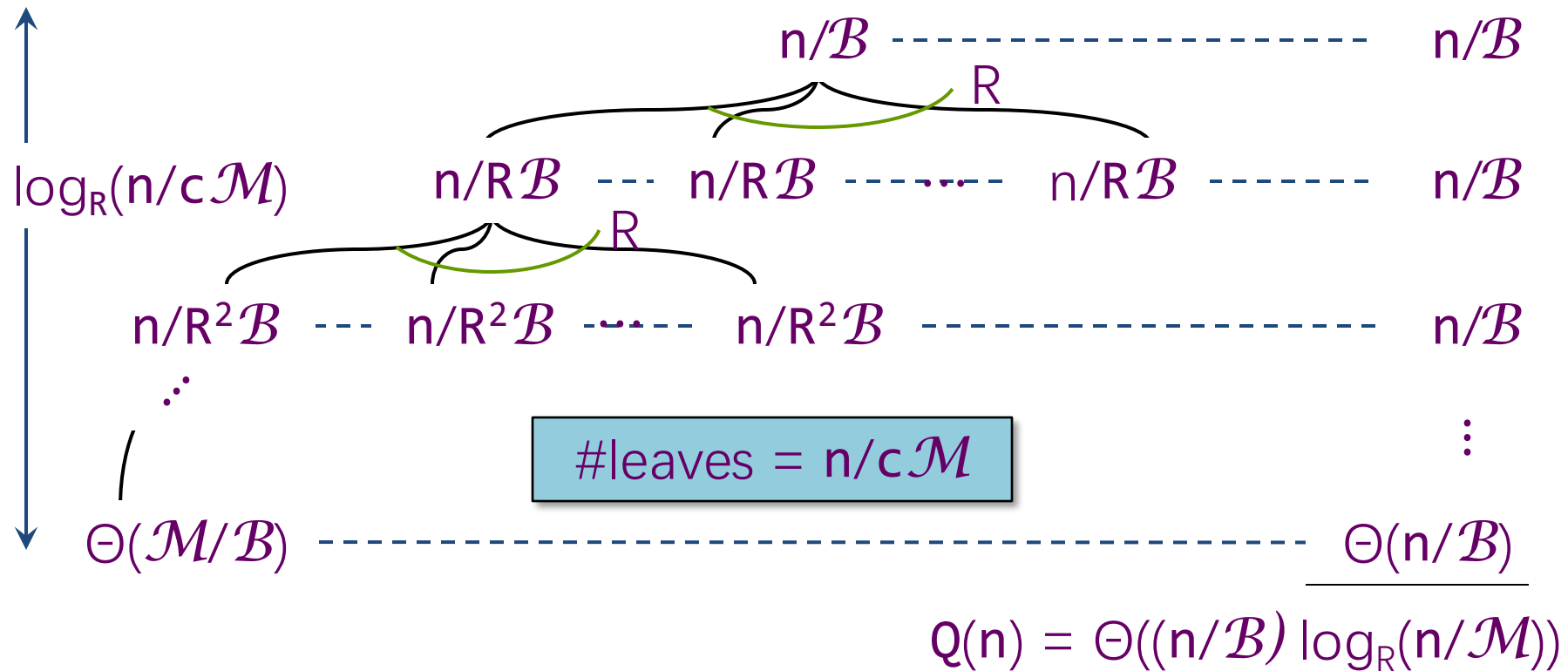
R -way merge sort

$$Q(n) \leq \begin{cases} \Theta(n/\mathcal{B}) & \text{if } n < c\mathcal{M}; \\ R \cdot Q(n/R) + \Theta(n/\mathcal{B}) & \text{otherwise.} \end{cases}$$

Cache Analysis

$$Q(n) \leq \begin{cases} \Theta(n/\mathcal{B}) & \text{if } n < c\mathcal{M}; \\ R \cdot Q(n/R) + \Theta(n/\mathcal{B}) & \text{otherwise.} \end{cases}$$

Recursion tree



Tuning the Voodoo Parameter

We have

$$Q(n) = \Theta((n/\mathcal{B}) \log_R(n/\mathcal{M})) ,$$

which decreases as $R < c\mathcal{M}/\mathcal{B}$ increases. Choosing R as big as possible yields

$$R = \Theta(\mathcal{M}/\mathcal{B}) .$$

By the tall-cache assumption ($\mathcal{B}^2 < c\mathcal{M}$) and the fact that $\log_{\mathcal{M}}(n/\mathcal{M}) = \Theta((\lg n)/\lg \mathcal{M})$, we have

$$\begin{aligned} Q(n) &= \Theta((n/\mathcal{B}) \log_{\mathcal{M}/\mathcal{B}}(n/\mathcal{M})) \\ &= \Theta((n/\mathcal{B}) \log_{\mathcal{M}}(n/\mathcal{M})) \\ &= \Theta((n \lg n)/\mathcal{B} \lg \mathcal{M}) . \end{aligned}$$

Hence, we have $W(n)/Q(n) \approx \Theta(\mathcal{B} \lg \mathcal{M})$.

Multiway versus Binary Merge Sort

We have

$$Q_{\text{multiway}}(n) = \Theta((n \lg n) / \mathcal{B} \lg \mathcal{M})$$

versus

$$\begin{aligned} Q_{\text{binary}}(n) &= \Theta((n/\mathcal{B}) \lg(n/\mathcal{M})) \\ &= \Theta((n \lg n) / \mathcal{B}) , \end{aligned}$$

as long as $n \gg \mathcal{M}$, because then $\lg(n/\mathcal{M}) \approx \lg n$.

Thus, multiway merge sort saves a factor of $\Theta(\lg \mathcal{M})$ in cache misses.

Example (ignoring constants)

- L1-cache: $\mathcal{M} = 2^{15} \Rightarrow 15\times$ savings.
- L2-cache: $\mathcal{M} = 2^{18} \Rightarrow 18\times$ savings.
- L3-cache: $\mathcal{M} = 2^{23} \Rightarrow 23\times$ savings.

Optimal Cache-Oblivious Sorting

Funnelsort [FLPR99]

1. Recursively sort $n^{1/3}$ groups of $n^{2/3}$ items.
2. Merge the sorted groups with an $n^{1/3}$ -funnel.

A k -funnel merges k^3 items in k sorted lists, incurring at most

$$\Theta(k + (k^3/\mathcal{B})(1 + \log_{\mathcal{M}} k))$$

cache misses. Thus, funnelsort incurs

$$\begin{aligned} Q(n) &\leq n^{1/3}Q(n^{2/3}) + \Theta(n^{1/3} + (n/b)(1 + \log_{\mathcal{M}} n)) \\ &= \Theta(1 + (n/\mathcal{B})(1 + \log_{\mathcal{M}} n)), \end{aligned}$$

cache misses, which is asymptotically optimal [AV88].

Construction of a k -funnel

